

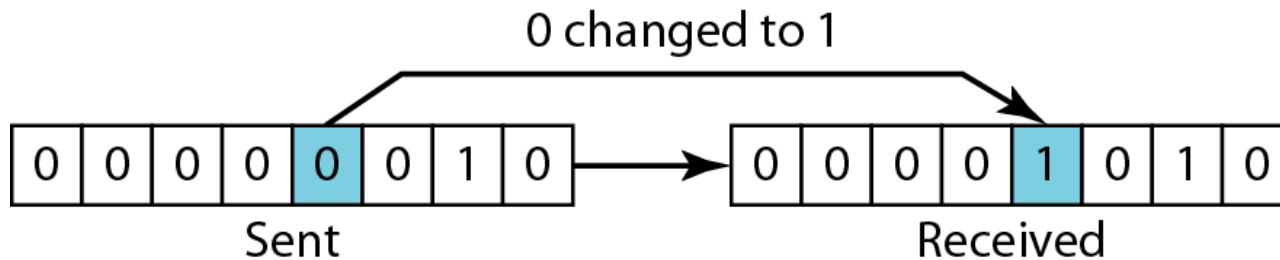
Ramaiah Institute of Technology

Department of AI & DS

Data Communication & Networking

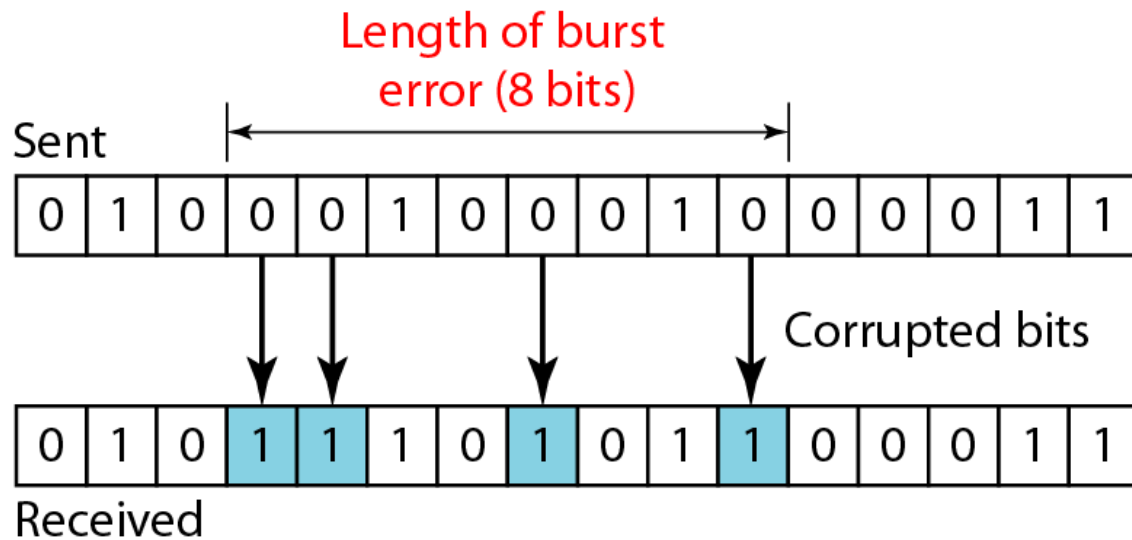
Unit 3

Figure 10.1 *Single-bit error*



Data can be corrupted during transmission.
Some applications require that errors be detected and corrected.
In single bit error only 1 bit of data is changed

Figure 10.2 *Burst error of length 8*

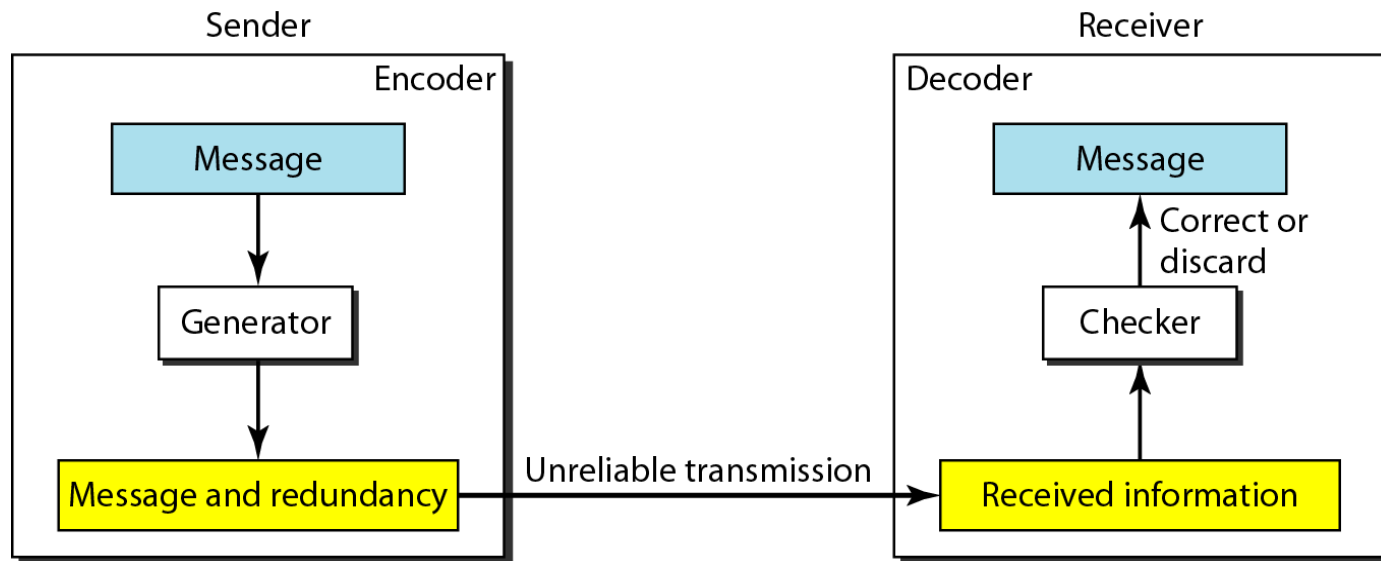


A burst error means that 2 or more bits in the data unit have changed.

To detect or correct errors, we need to send extra (redundant) bits with data.

Figure 10.3 *The structure of encoder and decoder*

To detect or correct errors, we need to send some extra bits with our data. These redundant bits are added by the sender and removed by the receiver.



Detection versus Correction

- **Error Detection:** Check if any error has occurred. The answer is a simple yes or no. We are not even interested in the number of corrupted bits.
- **Error Correction:** Need to know the exact number of bits that are corrupted and, more importantly, their location in the message.
- **The correction of errors is more difficult than the detection.**

Coding

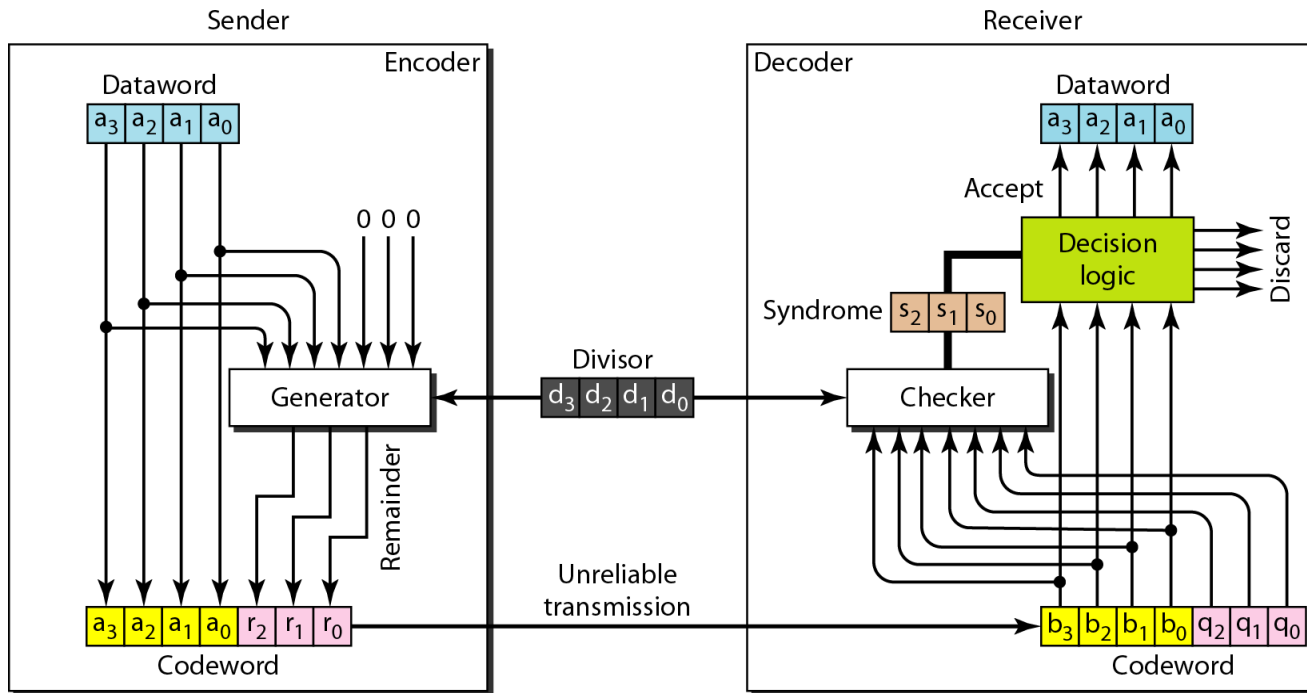
- Redundancy is achieved through various coding schemes.
- The sender adds redundant bits through a process that creates a relationship between the redundant bits and the actual data bits.
- The receiver checks the relationships between the two sets of bits to detect errors.
- The ratio of redundant bits to data bits and the robustness of the process are important factors in any coding scheme.

Cyclic Redundancy Check (CRC)

Table 10.3 A CRC code with $C(7, 4)$

<i>Dataword</i>	<i>Codeword</i>	<i>Dataword</i>	<i>Codeword</i>
0000	0000000	1000	1000101
0001	0001011	1001	1001110
0010	0010110	1010	1010011
0011	0011101	1011	1011000
0100	0100111	1100	1100010
0101	0101100	1101	1101001
0110	0110001	1110	1110100
0111	0111010	1111	1111111

Figure 10.14 *CRC encoder and decoder*



Cyclic Redundancy Check (CRC)

Encoder

- The dataword has k bits (4 here); the codeword has n bits (7 here).
 - The size of the dataword is augmented by adding $n - k$ (3 here) 0s to the right-hand side of the word. The n -bit result is fed into the generator.
 - The generator uses a divisor of size $n - k + 1$ (4 here), predefined and agreed upon.
 - The generator divides the augmented dataword by the divisor (modulo-2 division).
 - The quotient of the division is discarded; the remainder ($r_2r_1r_0$) is appended to the dataword to create the codeword.
-

Cyclic Redundancy Check (CRC)

Decoder

- The decoder receives the codeword (possibly corrupted in transition).
 - A copy of all n bits is fed to the checker, which is a replica of the generator.
 - The remainder produced by the checker is a syndrome of $n - k$ (3 here) bits, which is fed to the decision logic analyzer.
 - The analyzer has a simple function. If the syndrome bits are all 0s, the 4 leftmost bits of the codeword are accepted as the dataword (interpreted as no error); otherwise, the 4 bits are discarded (error)
-

Figure 10.15 *Division in CRC encoder*

The encoder takes a dataword and augments it with $n - k$ number of 0s. It then divides the augmented dataword by the divisor

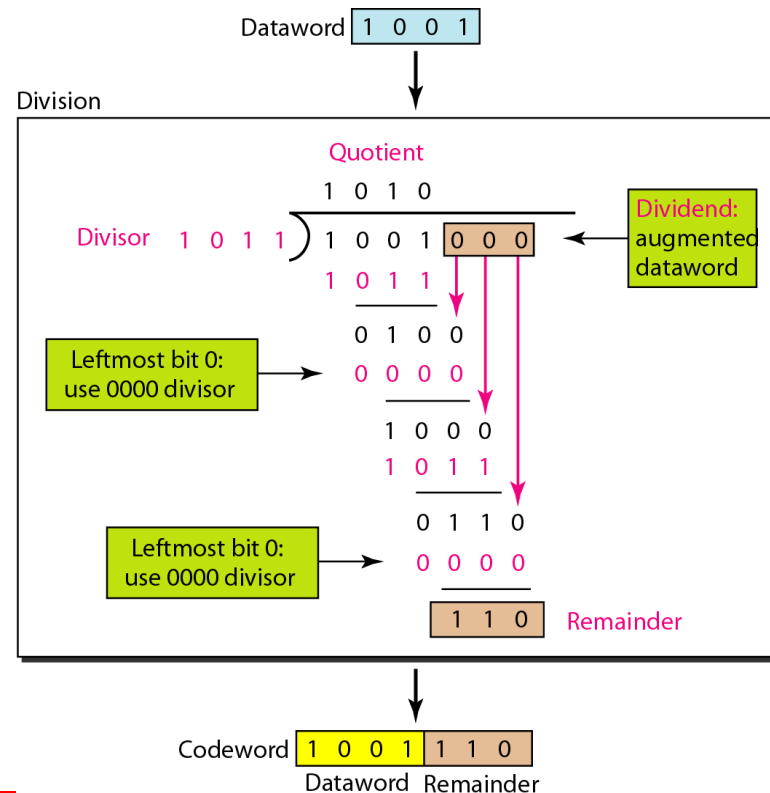
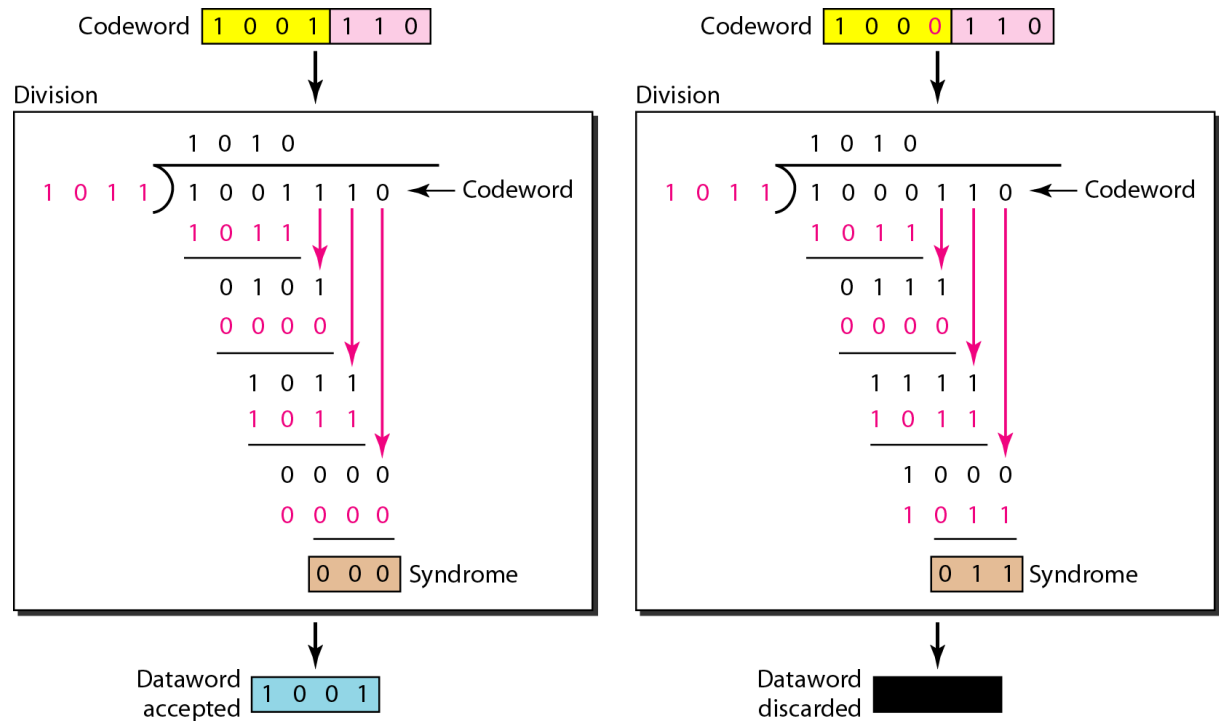


Figure 10.16 *Division in the CRC decoder for two cases*

- The codeword can change during transmission.
- The decoder does the same division process as the encoder. The remainder of the division is the syndrome.
- If the syndrome is all 0s, there is no error with a high probability; the dataword is separated from the received codeword and accepted.
- Otherwise, everything is discarded.



CRC Polynomial

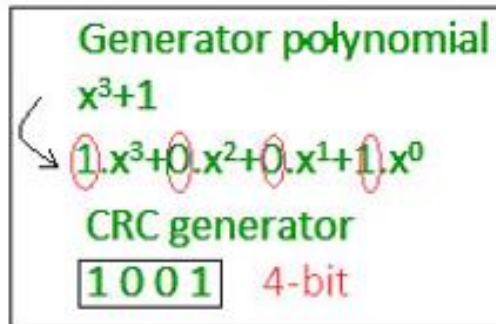
The divisor in a cyclic code is normally called the generator polynomial or simply the generator.

Table 10.4 *Standard polynomials*

<i>Name</i>	<i>Polynomial</i>	<i>Used in</i>
CRC-8	$x^8 + x^2 + x + 1$ 100000111	ATM header
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x^2 + 1$ 11000110101	ATM AAL
CRC-16	$x^{16} + x^{12} + x^5 + 1$ 10001000000100001	HDLC
CRC-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$ 100000100110000010001110110110111	LANs

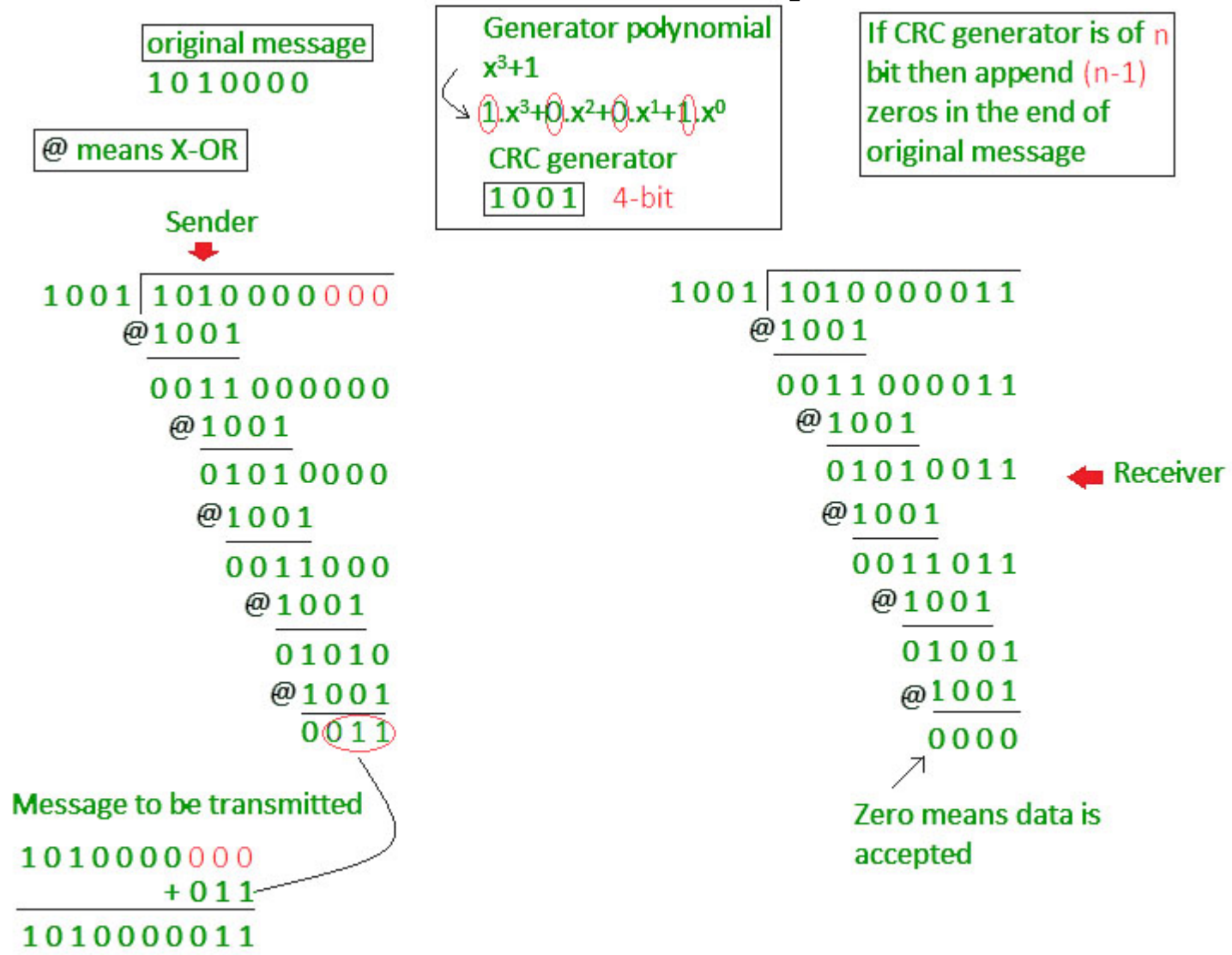
CRC Example

1. Given the dataword 1010000 and the divisor x^3+1 , show the generation of the CRC codeword at the sender site (using binary division) and compute the decoding on receiving side to check errors.



If CRC generator is of n bit then append $(n-1)$ zeros in the end of original message

CRC Example



CRC Example

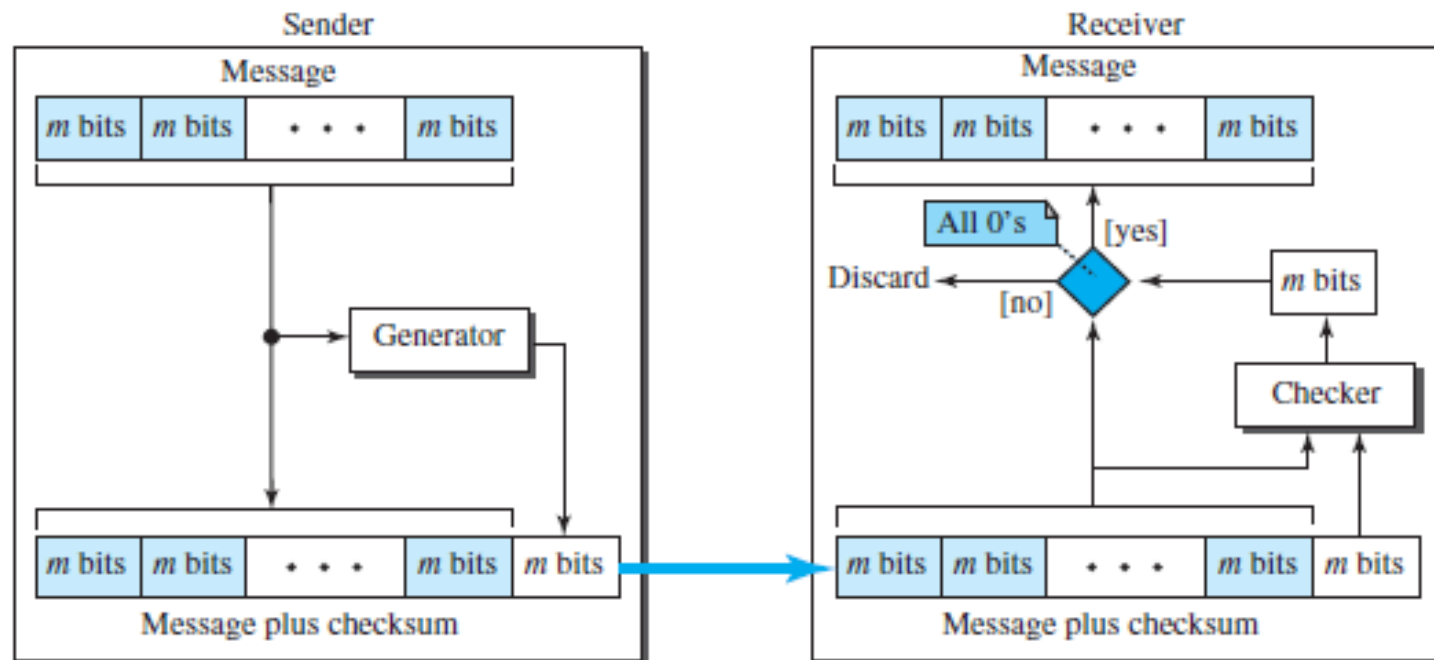
1. Given the dataword 1001 and the divisor 1011, show the process of encoding and decoding at the sender site and receiver site using polynomial division.
2. Given the dataword 101001111 and the divisor 10111, show the generation of the CRC codeword at the sender site (using binary division) and compute the decoding on receiving side to check errors.
3. A bit stream 10011101 is transmitted using the standard CRC method. The generator polynomial is x^3+1 .
 - i. What is the actual bit string transmitted?
 - ii. Suppose the third bit from the left is inverted during transmission. How will receiver detect this error?
4. Given the dataword 101001111 and the divisor 10111, show the generation of the CRC codeword at the sender site (using binary division).

Checksum

- Checksum is an error-detecting technique that can be applied to a **message of any length**.
- In the Internet, the checksum technique is mostly **used at the network and transport layer** rather than the data-link layer.
- **At the source, the message is first divided into m-bit units.**
- The generator then creates an **extra m-bit unit** called the checksum, which is sent with the message.
- At the destination, the checker **creates a new checksum** from the combination of the message and sent checksum.
- If the new **checksum is all 0s**, the message is accepted; otherwise, the message is discarded.
- The checksum unit is not necessarily added at the end of the message; it can be inserted in the **middle of the message**.

Checksum

Figure 10.15 Checksum



Checksum

Example

Suppose that the sender wants to send 4 frames each of 8 bits, where the frames are 11001100, 10101010, 11110000 and 11000011.

The sender adds the bits using 1s complement arithmetic. While adding two numbers using 1s complement arithmetic, if there is a carry over, it is added to the sum.

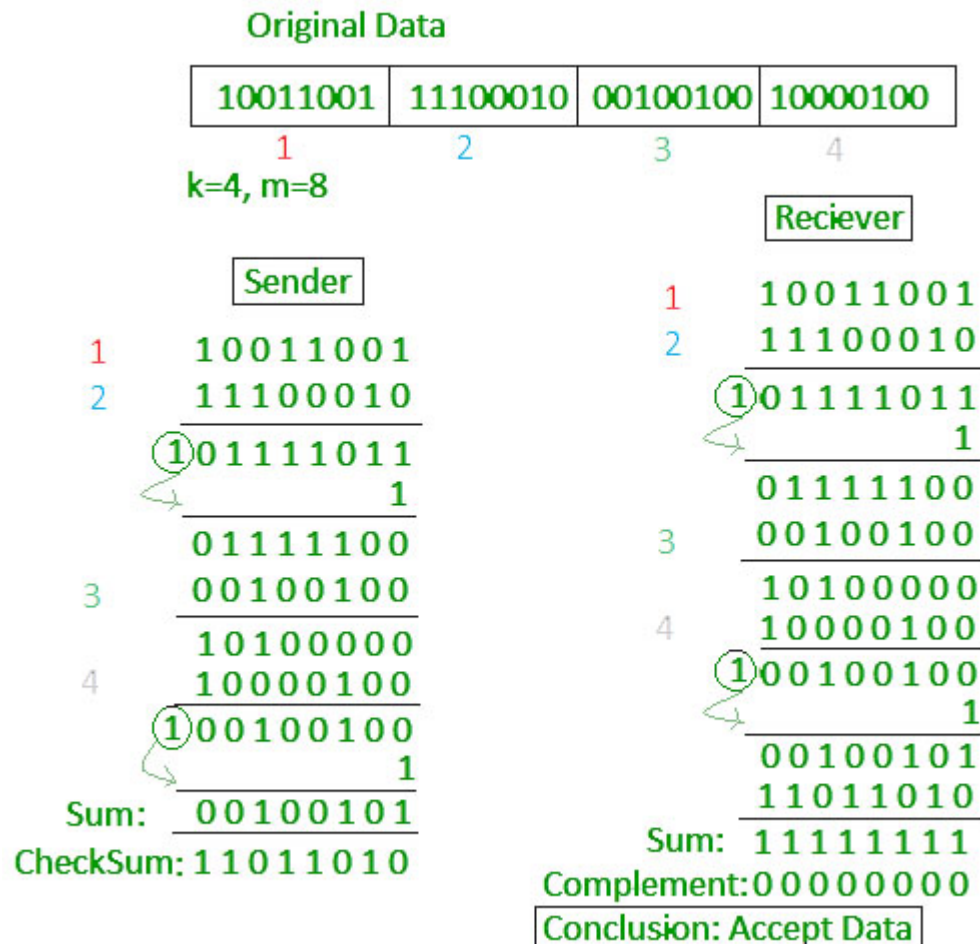
After adding all the 4 frames, the sender complements the sum to get the checksum, 11010011, and sends it along with the data frames.

The receiver performs 1s complement arithmetic sum of all the frames including the checksum. The result is complemented and found to be 0. Hence, the receiver assumes that no error has occurred.

Checksum

Sender's End	Receiver's End
Frame 1: 11001100	Frame 1: 11001100
Frame 2: + 10101010	Frame 2: + 10101010
Partial Sum: 1 01110110	Partial Sum: 1 01110110
+ 1	+ 1
01110111	01110111
Frame 3: + 11110000	Frame 3: + 11110000
Partial Sum: 1 01100111	Partial Sum: 1 01100111
+ 1	+ 1
01101000	01101000
Frame 4: + 11000011	Frame 4: + 11000011
Partial Sum: 1 00101011	Partial Sum: 1 00101011
+ 1	+ 1
Sum: 00101100	Sum: 00101100
Checksum: 11010011	Checksum: 11010011
	Sum: 11111111
	Complement: 00000000
	Hence accept frames.

Checksum



Checksum

Internet Checksum

Traditionally, the Internet has used a 16-bit checksum.

Table 10.5 *Procedure to calculate the traditional checksum*

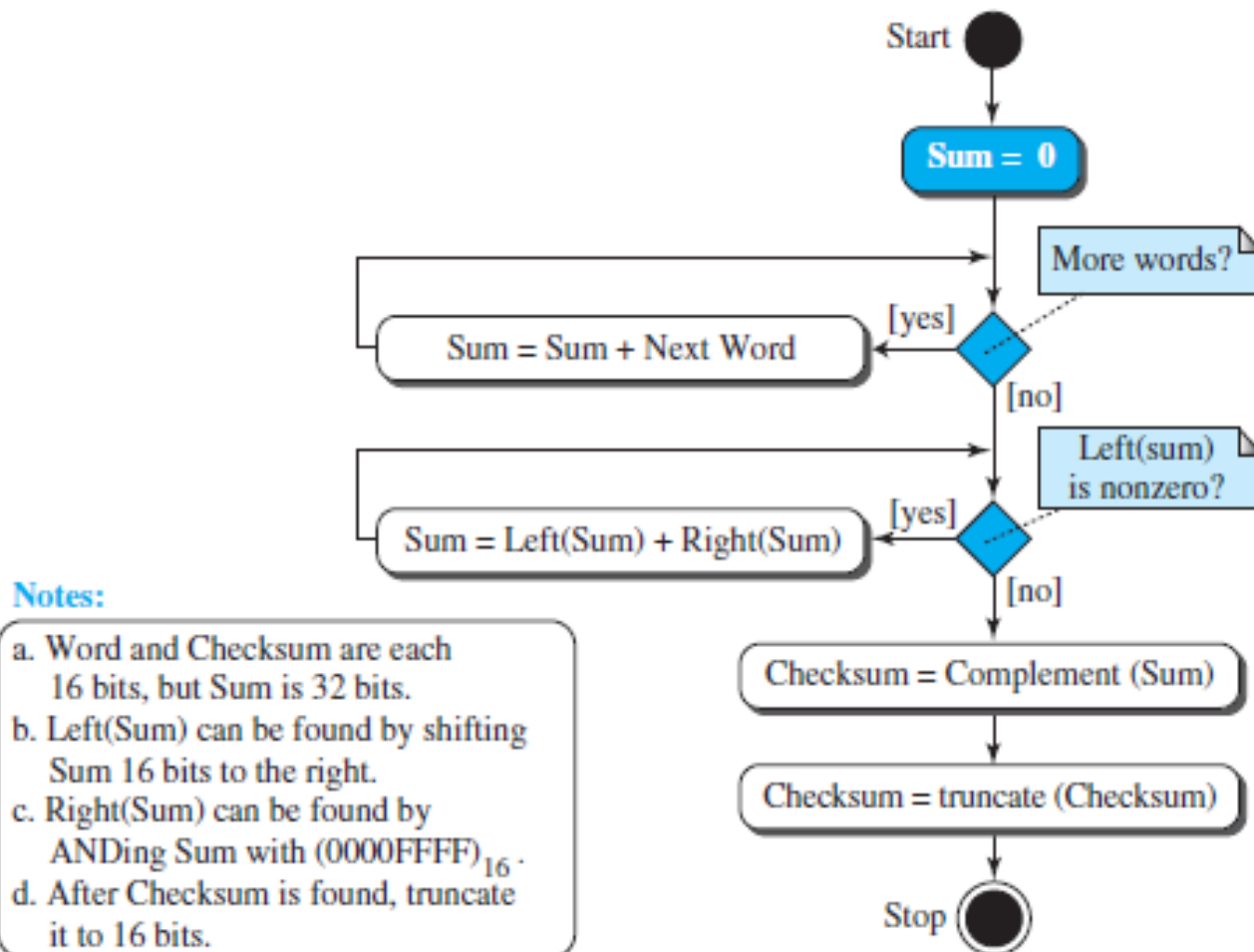
<i>Sender</i>	<i>Receiver</i>
1. The message is divided into 16-bit words. 2. The value of the checksum word is initially set to zero. 3. All words including the checksum are added using one's complement addition. 4. The sum is complemented and becomes the checksum. 5. The checksum is sent with the data.	1. The message and the checksum are received. 2. The message is divided into 16-bit words. 3. All words are added using one's complement addition. 4. The sum is complemented and becomes the new checksum. 5. If the value of the checksum is 0, the message is accepted; otherwise, it is rejected.

Checksum

Internet Checksum

Traditionally, the Internet has used a 16-bit checksum.

Figure 10.17 *Algorithm to calculate a traditional checksum*



Chapter 3

Network Layer : Logical Addressing

Error detection & correction: Cyclic codes – CRC, Polynomials, Checksum.

Network layer: Logical addressing - IPV4 addresses, Address space, notations, classful and classless addressing with problem solving, NAT, IPV6 addresses; IPV6 addresses; Network layer: Internet protocol - IPV4 datagram, fragmentation, checksum and options; IPV6 packet format, advantages and extension headers; Transition from IPV4 to IPV6.

IPv4 Addresses

Address Space

Notations

Classful Addressing

Classless Addressing

Network Address Translation (NAT)

IPv4 Addresses

Physical / MAC Address: 48-bit (6-byte) physical address written as 12 hexadecimal digits; every byte (2 hexadecimal digits) is separated by a colon, as shown below:

07:01:02:01:2C:4B

A 6-byte (12 hexadecimal digits) physical address.

IPv4 Addresses

*An **IPv4 address** is a **32-bit** address that uniquely and universally defines the connection of a device (for example, a computer or a router) to the Internet.*

- IPv4 addresses are **unique and universal**.
- They are unique as each address defines **one, and only one, connection to the Internet**.
- Two devices on the Internet can never have the same address at the same time.

IPv4 Addresses: Address Space

- An **address space** is the total number of addresses used by the protocol.
- If a protocol uses **N bits** to define an address, the **address space** is 2^N

The address space of IPv4 is
 2^{32} or 4,294,967,296.

IPv4 Addresses: Notations

- There are two prevalent notations to show an IPv4 address:
 1. Binary notation
 2. Dotted decimal notation

IPv4 Addresses: Notations

1. Binary Notation

- In binary notation, the IPv4 address is displayed as 32 bits.
- Each octet is often referred to as a byte.
- So an IPv4 address referred to as a 32-bit address or a 4-byte address.

01110101 10010101 00011101 00000010

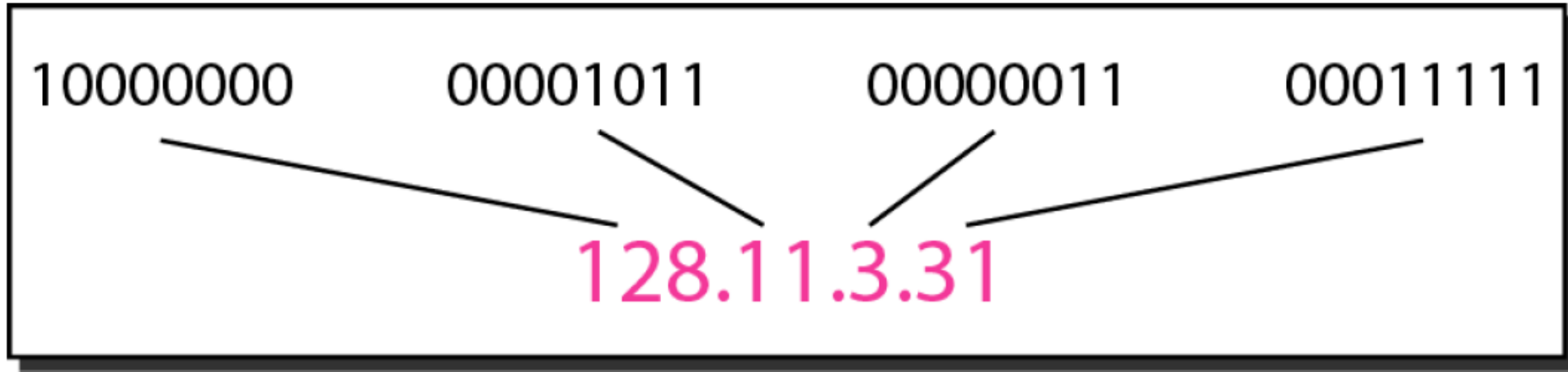
IPv4 Addresses: Notations

2. Dotted-Decimal Notation

- To make the IPv4 address more **compact and easier to read**, Internet addresses are usually written in decimal form with a decimal point (dot) separating the bytes.
- The following is the dotted-decimal notation of the above address:

117.149.29.2

IPv4 Addresses: Notations



IPv4 Addresses: Notations

Change the following IPv4 addresses from binary notation to dotted-decimal notation.

a. 10000001 00001011 00001011 11101111

b. 11000001 10000011 00011011 11111111

Solution

We replace each group of 8 bits with its equivalent decimal number (see Appendix B) and add dots for separation.

a. 129.11.11.239

b. 193.131.27.255

IPv4 Addresses: Notations

Change the following IPv4 addresses from dotted-decimal notation to binary notation.

a. 111.56.45.78

b. 221.34.7.82

Solution

We replace each decimal number with its binary equivalent (see Appendix B).

a. 01101111 00111000 00101101 01001110

b. 11011101 00100010 00000111 01010010

IPv4 Addresses: Notations

Find the error, if any, in the following IPv4 addresses.

- a. 111.56.045.78
- b. 221.34.7.8.20
- c. 75.45.301.14
- d. 11100010.23.14.67

Solution

- a.*** There must be no leading zero (045).
- b.*** There can be no more than four numbers.
- c.*** Each number needs to be less than or equal to 255.
- d.*** A mixture of binary notation and dotted-decimal notation is not allowed.

IPv4 Addresses: Classful Addressing

- IPv4 addressing, uses the concept of **classes**.
- This **architecture** is called classful addressing.
- In classful addressing, the address space is divided into five classes: **A, B, C, D, and E**.
- Each class **occupies some part** of the address space.

	First byte	Second byte	Third byte	Fourth byte
Class A	0			
Class B	10			
Class C	110			
Class D	1110			
Class E	1111			

a. Binary notation

	First byte	Second byte	Third byte	Fourth byte
Class A	0–127			
Class B	128–191			
Class C	192–223			
Class D	224–239			
Class E	240–255			

b. Dotted-decimal notation

IPv4 Addresses: Classful Addressing

Find the class of each address.

- a.** 000000001 00001011 00001011 11101111
- b.** 11000001 10000011 00011011 11111111
- c.** 14.23.120.8
- d.** 252.5.15.111

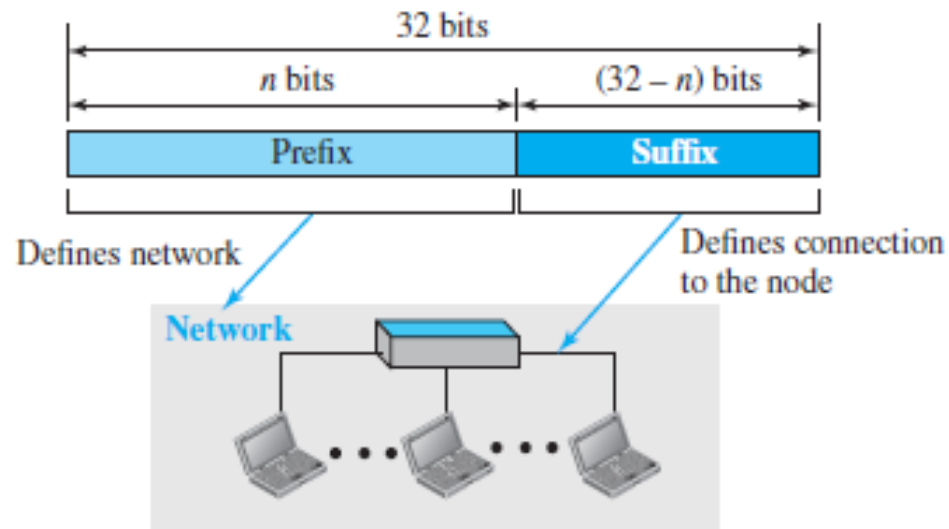
Solution

- a.** *The first bit is 0. This is a class A address.*
- b.** *The first 2 bits are 1; the third bit is 0. This is a class C address.*
- c.** *The first byte is 14; the class is A.*
- d.** *The first byte is 252; the class is E.*

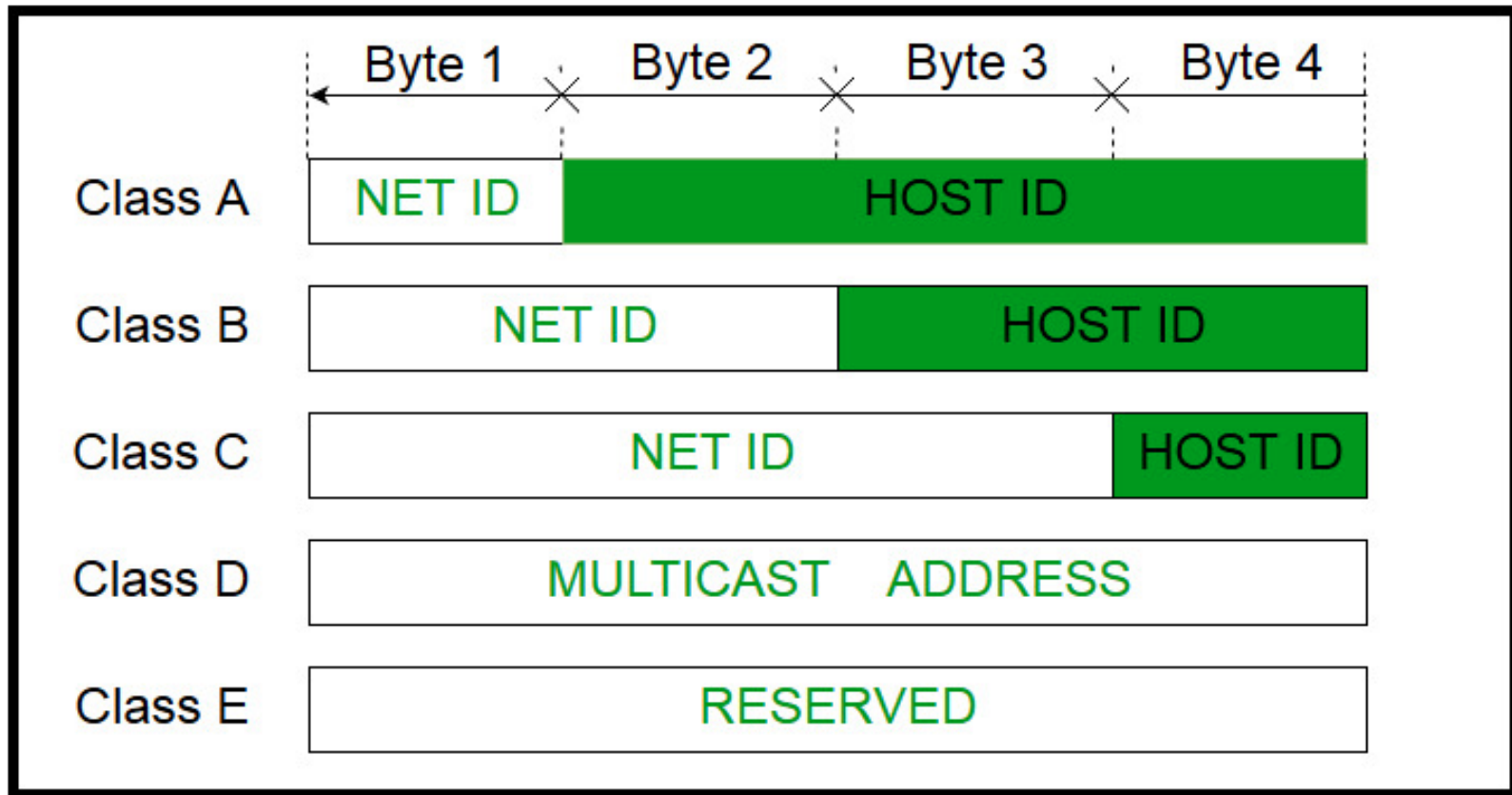
IPv4 Addresses: Classful Addressing

- Each of these classes has a **valid range of IP addresses**.
- **Classes D and E** are reserved for **multicast and experimental** purposes.
- The IPv4 address is divided into **two parts**:
 - **Network ID**
 - **Host ID**
- The class of IP address is used to determine the bits used for network ID and host ID and the **number of total networks and hosts** possible in that particular class.
- Each **ISP or network administrator** assigns an IP address to each device that is connected to its network.

Hierarchy in addressing

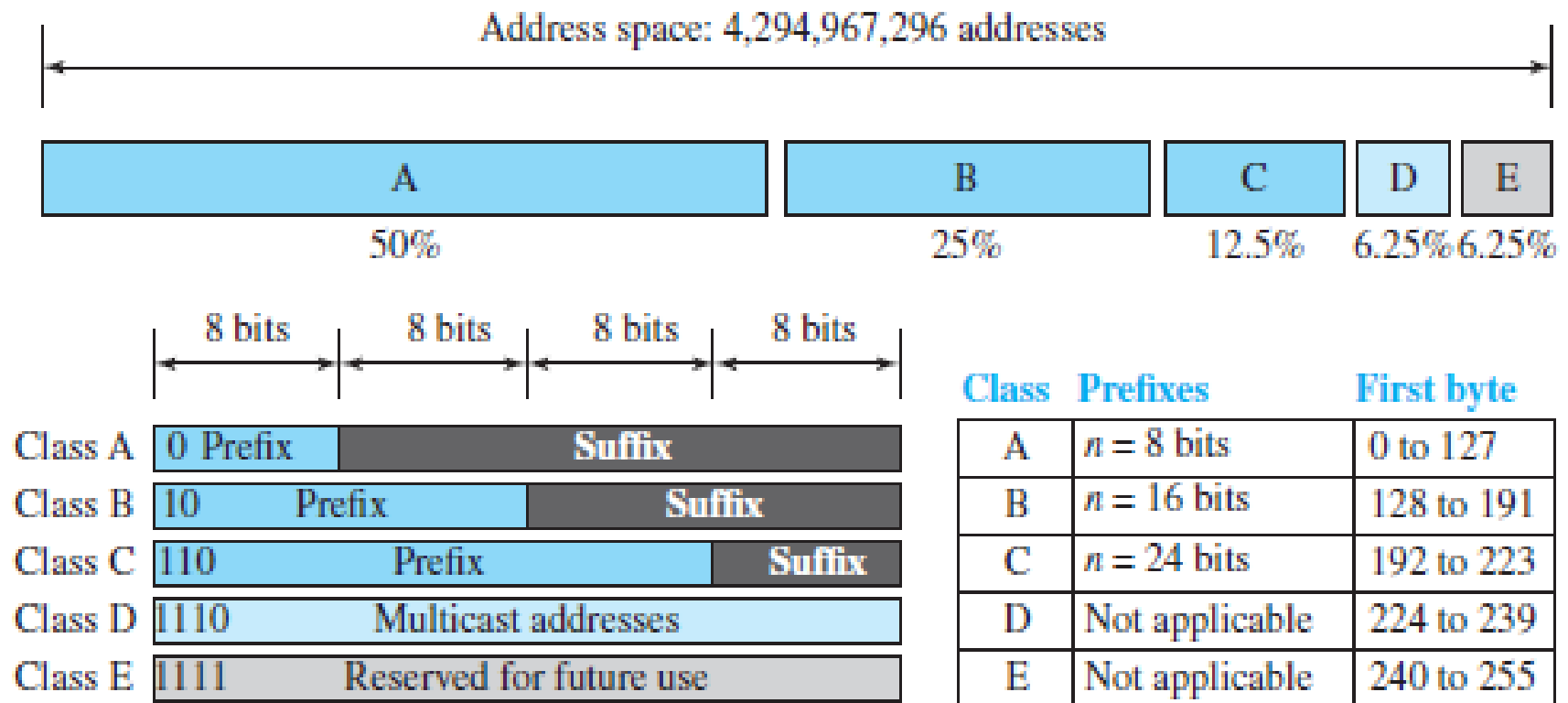


IPv4 Addresses: Classful Addressing



IPv4 Addresses: Classful Addressing

Figure 18.18 *Occupation of the address space in classful addressing*



IPv4 Addresses: Classful Addressing

- IP addresses are **globally managed** by Internet Assigned Numbers Authority(IANA) and Regional Internet Registries(RIR).
- While finding the total number of host IP addresses, 2 IP addresses are not counted and are therefore, decreased from the total count because the
- **First IP address of any network is the network number and whereas the last IP address is reserved for broadcast IP.**

IPv4 Addresses: Classful Addressing

- **Class A**

- IP addresses of class A are assigned to the networks that contain a large number of hosts.
 - The network ID is 8 bits long.
 - The host ID is 24 bits long.

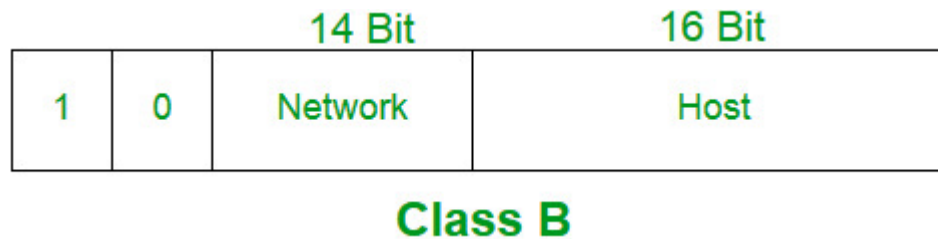


Class A

- The default subnet mask for Class A is 255.x.x.x. Therefore, class A has a total of:
 - $2^7 = 128$ network address
 - $2^{24} - 2 = 16,777,214$ host ID
- IP addresses belonging to class A ranges from **0.0.0.0 – 127.255.255.255.**

IPv4 Addresses: Classful Addressing

- **Class B**
- IP address belonging to class B is assigned to networks that range from medium-sized to large-sized networks.
 - The network ID is 16 bits long.
 - The host ID is 16 bits long.



- The default subnet mask for class B is 255.255.x.x. Class B has a total of:
 - $2^{14} = 16384$ network address
 - $2^{16} - 2 = 65534$ host address
- IP addresses belonging to class B ranges from **128.0.0.0 – 191.255.255.255.**

IPv4 Addresses: Classful Addressing

- **Class C**
- IP addresses belonging to class C are assigned to small-sized networks.
 - The network ID is 24 bits long.
 - The host ID is 8 bits long.

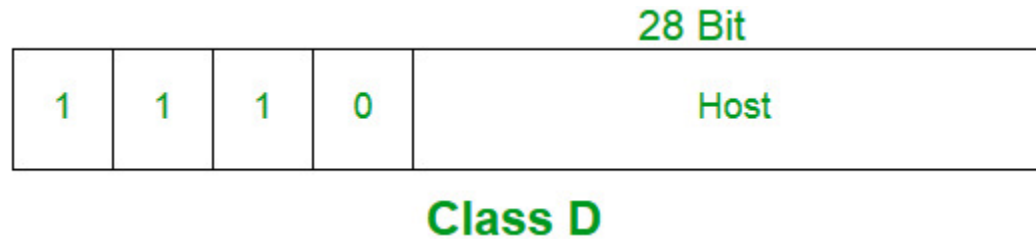


Class C

- The default subnet mask for class C is 255.255.255.x. Class C has a total of:
 - $2^{21} = 2097152$ network address
 - $2^8 - 2 = 254$ host address
- IP addresses belonging to class C range from
192.0.0.0 – 223.255.255.255.

IPv4 Addresses: Classful Addressing

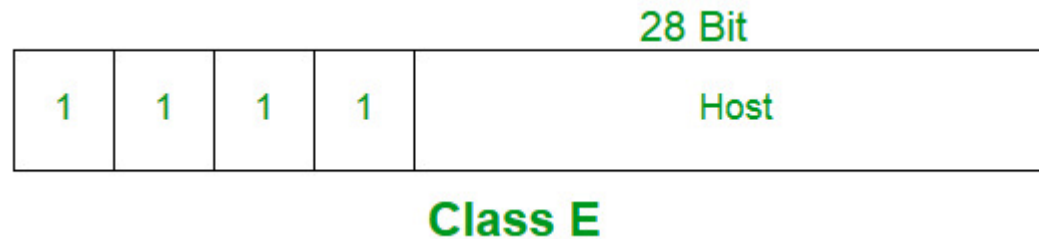
- **Class D**
- IP address belonging to class D is reserved for multi-casting.



- Class D does not possess any subnet mask.
- IP addresses belonging to class D range from **224.0.0.0 – 239.255.255.255.**

IPv4 Addresses: Classful Addressing

- **Class E**
- IP addresses belonging to class E are reserved for experimental and research purposes.



- IP addresses of class E range from 240.0.0.0 – 255.255.255.254. This class doesn't have any subnet mask.
- The higher-order bits of the first octet of class E are always set to 1111.

IPv4 Addresses: Classful Addressing

- **Range of Special IP Addresses**

`169.254.0.0 - 169.254.0.16` : Link-local addresses

`127.0.0.0 - 127.255.255.255` : Loop-back addresses

`0.0.0.0 - 0.0.0.8`: used to communicate within the current network.

IPv4 Addresses: Classful Addressing

Drawback : Address Depletion

The classful addressing has **become obsolete**, because the addresses were not distributed properly.

Class A.

Assigned to only 128 organizations in the world, but each organization needs to have a single network with 16,777,216 nodes. But there may be only a few organizations that are this large, most of the addresses in this class were wasted (unused).

Class B

Addresses were designed for midsize organizations, but many of the addresses in this class also remained unused.

Class C

Addresses have a completely different flaw in design. The number of addresses that can be used in each network (256) was so small that most companies were not comfortable using a block in this address class.

Class E

Addresses were almost never used, wasting the whole class.

IPv4 Addresses: Classful Addressing

Subnetting and Supernetting

To alleviate address depletion, two strategies were proposed : Subnetting and Supernetting.

Subnetting:

- A **class A or class B** block is divided into several subnets.
- If an organization was granted a large block in class A or B, it could divide the addresses into several contiguous groups and assign each group to smaller networks (called subnets) or, in rare cases, share part of the addresses with neighbors.

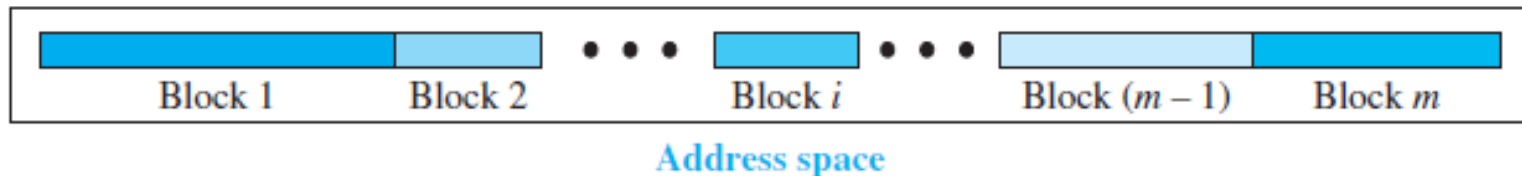
Supernetting:

- An organization can combine several **class C blocks** to create a larger range of addresses.
- For example, an organization that needs 1000 addresses can be granted four contiguous class C blocks. The organization can then use these addresses to create one supernetwork.

IPv4 Addresses: Classless Addressing

- In 1996, the Internet authorities announced a new architecture called classless addressing.
- In classless addressing, **variable-length blocks are used that belong to no classes.**
- We can have a block of 1 address, 2 addresses, 4 addresses, 128 addresses, and so on.
- In classless addressing, the whole address space is divided into variable length blocks.

Figure 18.19 *Variable-length blocks in classless addressing*

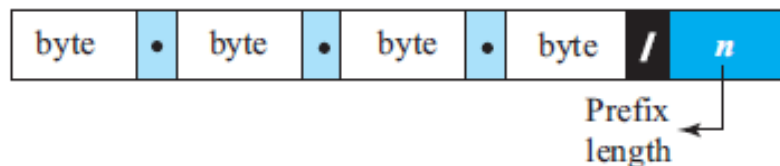


IPv4 Addresses: Classless Addressing

Prefix Length: Slash Notation

- The prefix in an address defines the block (network); the suffix defines the node (device).
- Unlike classful addressing, the prefix length in classless addressing is variable. We can have a prefix length that ranges from 0 to 32. The size of the network is inversely proportional to the length of the prefix. **A small prefix means a larger network; a large prefix means a smaller network.**
- Prefix length is not inherent in the address, need to separately give the length of the prefix.
- The prefix length, n , is added to the address, separated by a slash. The notation is informally referred to as **slash notation** and formally as **classless interdomain routing or CIDR**

Figure 18.20 *Slash notation (CIDR)*



Examples:
12.24.76.8/8
23.14.67.92/12
220.8.24.255/25

IPv4 Addresses: Classless Addressing

Prefix Length: Slash Notation

Figure 18.20 *Slash notation (CIDR)*

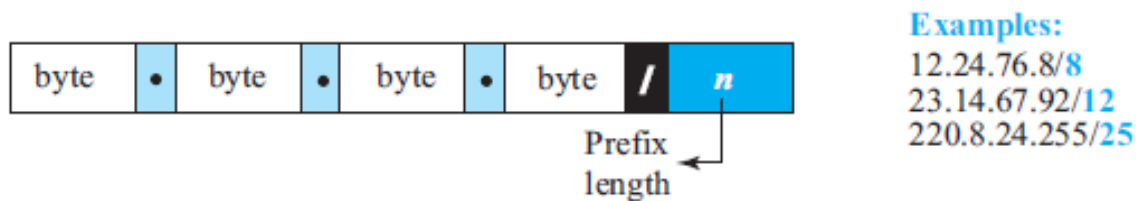
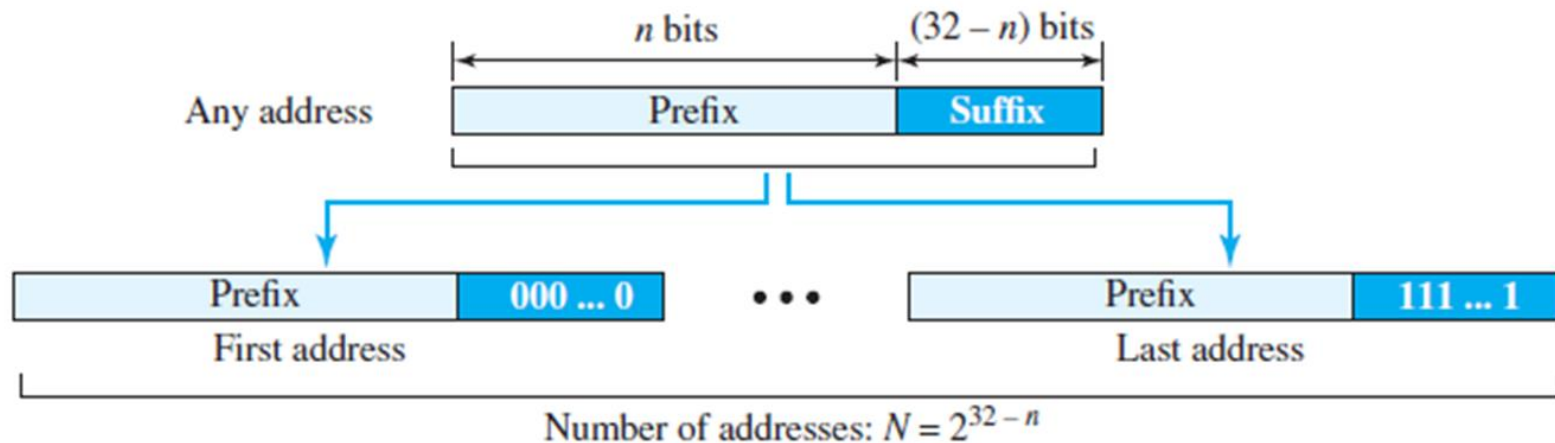


Figure 18.21 *Information extraction in classless addressing*

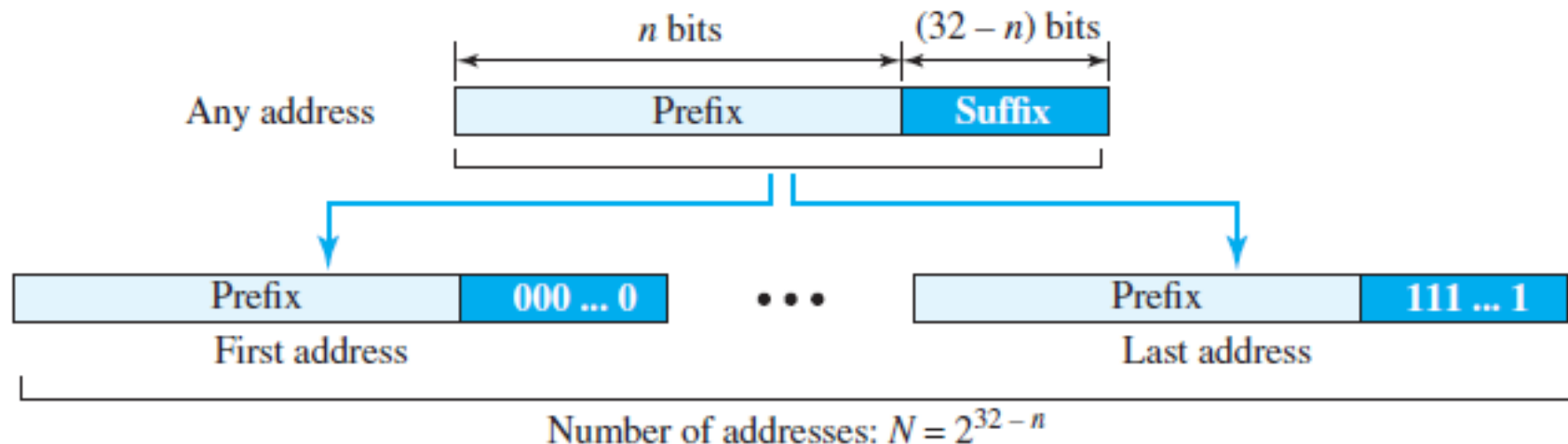


IPv4 Addresses: Classless Addressing

Extracting Information from an Address

- Given any address in the block, extract three information's:
 - The number of addresses:** Found as $N = 2^{32-n}$.
 - The first address in the block:** Keep the n leftmost bits and set the $(32 - n)$ rightmost bits all to 0s.
 - The last address:** Keep the n leftmost bits and set the $(32 - n)$ rightmost bits all to 1s.

Figure 18.21 *Information extraction in classless addressing*



IPv4 Addresses: Classless Addressing

Example 18.1

A classless address is given as 167.199.170.82/27. We can find the above three pieces of information as follows. The number of addresses in the network is $2^{32-n} = 2^5 = 32$ addresses.

The first address can be found by keeping the first 27 bits and changing the rest of the bits to 0s.

Address: 167.199.170.82/27	10100111	11000111	10101010	01010010
First address: 167.199.170.64/27	10100111	11000111	10101010	01000000

The last address can be found by keeping the first 27 bits and changing the rest of the bits to 1s.

Address: 167.199.170.82/27	10100111	11000111	10101010	01011111
Last address: 167.199.170.95/27	10100111	11000111	10101010	01011111

IPv4 Addresses: Classless Addressing

Each of the following addresses belongs to a block.
Find the first and the last address in each block.

- a. 14.12.72.8/24**
- b. 200.107.16.17/18**
- c. 70.110.19.17/16**

IPv4 Addresses: Classless Addressing

Address Mask

- Another way to find the first and last addresses in the block is to use the address mask.
- The address mask is a 32-bit number in which the **n leftmost bits are set to 1s and the rest of the bits (32 – n) are set to 0s.**

Ex: 169.199.170.82/27

Mask is : 11111111 11111111 11111111 11100000

- The reason for defining a mask in this way is that it can be used by a **computer program to extract the information in a block, using the three bit-wise operations NOT, AND, and OR.**
 1. The number of addresses in the block $N = \text{NOT}(\text{mask}) + 1$.
 2. The first address in the block = (Any address in the block) AND (mask).
 3. The last address in the block = (Any address in the block) OR [(NOT (mask))].

IPv4 Addresses: Classless Addressing

Example 5.27

One of the addresses in a block is 167.199.170.82/27. Find the number of addresses in the network, the first address, and the last address.

Solution

The value of n is 27. The network mask has twenty-seven 1s and five 0s. It is 255.255.255.240.

a. The number of addresses in the network is $2^{32-n} = 2^{32-27} = 2^5 = 32$.

b. We use the AND operation to find the first address (network address). The first address is 167.199.170.64/27.

Address in binary:	10100111	11000111	10101010	01010010
Network mask:	11111111	11111111	11111111	11100000
First address:	10100111	11000111	10101010	01000000

c. To find the last address, we first find the complement of the network mask and then OR it with the given address: The last address is 167.199.170.95/27.

Address in binary:	10100111	11000111	10101010	01010010
Complement of network mask:	00000000	00000000	00000000	00011111
Last address:	10100111	11000111	10101010	01011111

IPv4 Addresses: Classless Addressing

Example 5.29

One of the addresses in a block is 110.23.120.14/20. Find the number of addresses, the first address, and the last address in the block.

Solution

The network mask is 255.255.240.0.

a. The number of addresses in the network is $2^{32-20} = 4096$.

b. To find the first address, we apply the first short cut to bytes 1, 2, and 4 and the second short cut to byte 3. The first address is 110.23.112.0/20.

Address:	110	.	23	.	120	.	14
Network mask:	255	.	255	.	240	.	0
First address (AND):	110	.	23	.	112	.	0

c. To find the last address, we apply the first short cut to bytes 1, 2, and 4 and the second short cut to byte 3. The OR operation is applied to the complement of the mask. The last address is 110.23.127.255/20.

Address:	110	.	23	.	120	.	14
Network mask:	0	.	0	.	15	.	255
Last address (OR):	110	.	23	.	127	.	255

IPv4 Addresses: Classless Addressing

Example 5.28

One of the addresses in a block is 17.63.110.114/24. Find the number of addresses, the first address, and the last address in the block.

Solution

The network mask is 255.255.255.0.

a. The number of addresses in the network is $2^{32 - 24} = 256$.

b. To find the first address, we use the short cut methods discussed early in the chapter.

Address:	17	.	63	.	110	.	114
Network mask:	255	.	255	.	255	.	0
First address (AND):	17	.	63	.	110	.	0

The first address is 17.63.110.0/24.

c. To find the last address, we use the complement of the network mask and the first short cut method we discussed before. The last address is 17.63.110.255/24.

Address:	17	.	63	.	110	.	114
Complement of the mask (NOT):	0	.	0	.	0	.	255
Last address (OR):	17	.	63	.	110	.	255

IPv4 Addresses: Classless Addressing

- In classless addressing, an address cannot define the block the address belongs to.
- For example, the address 230.8.24.56 can belong to many blocks.
- Some of them are shown below with the value of the prefix associated with that block.

Prefix length:16	→	Block:	230.8.0.0	to	230.8.255.255
Prefix length:20	→	Block:	230.8.16.0	to	230.8.31.255
Prefix length:26	→	Block:	230.8.24.0	to	230.8.24.63
Prefix length:27	→	Block:	230.8.24.32	to	230.8.24.63
Prefix length:29	→	Block:	230.8.24.56	to	230.8.24.63
Prefix length:31	→	Block:	230.8.24.56	to	230.8.24.57

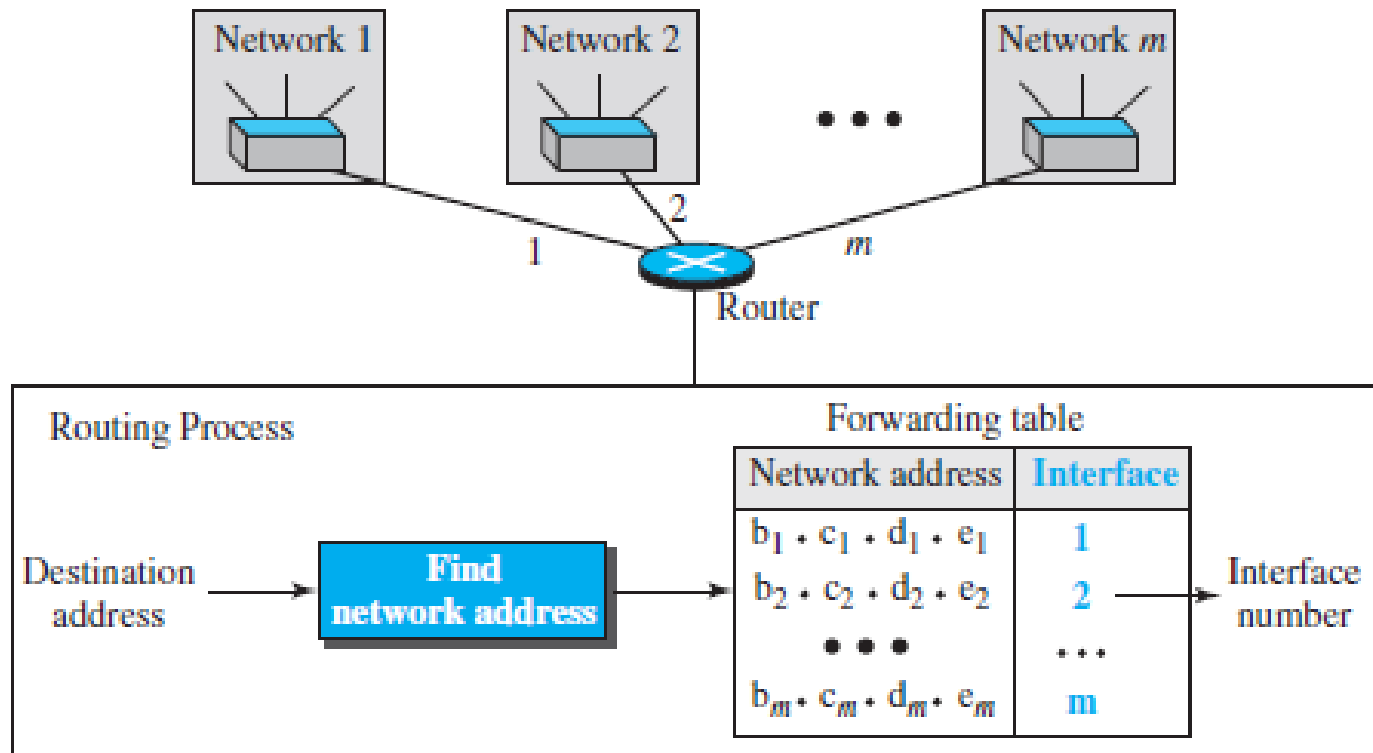
IPv4 Addresses: Classless Addressing

Network Address

- The network address, is particularly important because it is **used in routing a packet to its destination network.**
- Assume that an internet is made of **m** networks and a router with **m** interfaces.
- When a packet arrives at the router from any source host, the router needs to know to which network the packet should be sent, from which interface the packet should be sent out.
- Router consults its forwarding table to find the corresponding interface from which the packet should be sent out.
- **The network address is actually the identifier of the network; each network is identified by its network address.**

IPv4 Addresses: Classless Addressing

Figure 18.22 *Network address*



IPv4 Addresses: Classless Addressing

- **Block Allocation**

- How are the blocks allocated?
- Global authority called the Internet Corporation for Assigned Names and Numbers (ICANN)
- However, ICANN does not normally allocate addresses to individual Internet users.
- It assigns a large block of addresses to an ISP (or a larger organization that is considered an ISP in this case).
- For the proper operation of the CIDR, two restrictions need to be applied to the allocated block
 1. The number of requested addresses, N , *needs to be a power of 2*
 2. The requested block needs to be allocated where there are a **contiguous number of available addresses in the address space**

IPv4 Addresses: Classless Addressing

- An ISP has requested a block of 1000 addresses. Because 1000 is not a power of 2, 1024 addresses are granted.
- The prefix length is calculated as $n = 32 - \log_2 1024 = 22$.
- An available block, 18.14.12.0/22, is granted to the ISP.

IPv4 Addresses: Classless Addressing

- **Subnetting**

- An organization (or an ISP) that is granted a range of addresses may divide the range into several subranges and assign each subrange to a subnetwork (or subnet).
- Organization can create any levels of subnetwork.
- A subnetwork can be divided into several sub- subnetworks.
- A sub-subnetwork can be divided into several sub-sub-subnetworks, and so on.

IPv4 Addresses: Classless Addressing

- **N** - total number of addresses granted to the organization
- **n** - prefix length
- **Nsub**- the assigned number of addresses to each subnetwork
- **nsub** - the prefix length for each subnetwork
- Steps to guarantee the proper operation of the subnetworks
 - The number of addresses in each subnetwork should be a power of 2.
 - The prefix length for each subnetwork :
 - **$nsub = 32 - \log_2 Nsub$**
 - The starting address in each subnetwork should be divisible by the number of addresses in that subnetwork .

IPv4 Addresses: Classless Addressing

- An organization is granted a block of addresses with the beginning address 14.24.74.0/24. The organization needs to have three subblocks of addresses to use in its three subnets: one subblock of 10 addresses, one subblock of 60 addresses, and one subblock of 120 addresses. Design the subblocks.
 - There are $2^{32-24} = 256$ addresses in this block.
 - The **first address is 14.24.74.0/24;**
 - the **last address is 14.24.74.255/24.**
 - To satisfy the third requirement, we assign addresses to subblocks, starting with the largest and ending with the smallest one.

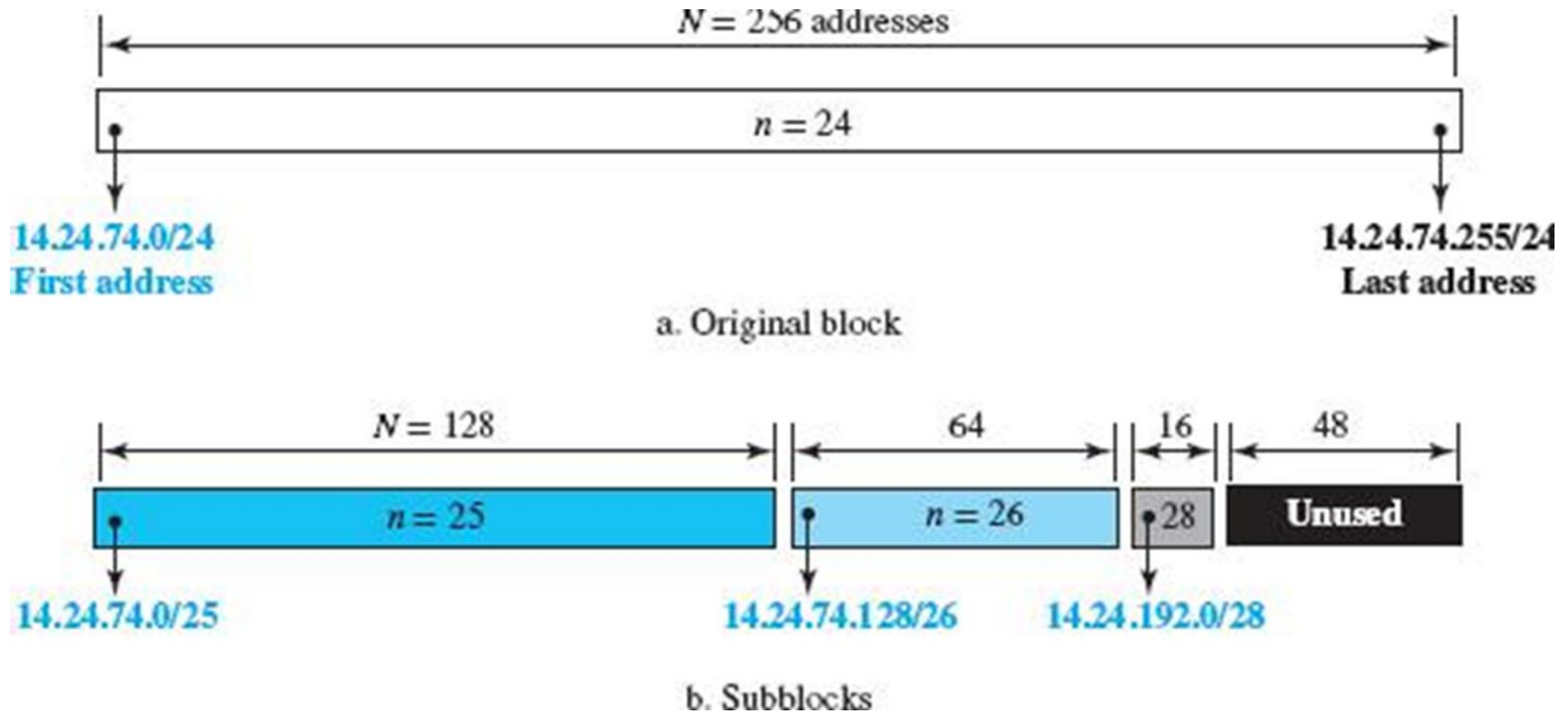
IPv4 Addresses: Classless Addressing

- **a. The number of addresses in the largest subblock, which requires 120 addresses, is not a power of 2.**
 - We allocate 128 addresses.
 - The subnet mask for this subnet can be found as $n1 = 32 - \log_2 128 = 25$.
 - *The first* address in this block is 14.24.74.0/25;
 - the last address is 14.24.74.127/25.
- **b. The number of addresses in the second largest subblock, which requires 60 addresses, is not a power of 2 either.**
 - We allocate 64 addresses
 - The subnet mask for this subnet can be found as $n2 = 32 - \log_2 64 = 26$.
 - *The first* address in this block is 14.24.74.128/26;
 - the last address is 14.24.74.191/26.

IPv4 Addresses: Classless Addressing

- **c.** The number of addresses in the smallest subblock, which requires 10 addresses, is not a power of 2 either.
 - allocate 16 addresses.
 - The subnet mask for this subnet can be found as $n3 = 32 - \log_2 16 = 28$.
 - *The* first address in this block is 14.24.74.192/28;
 - the last address in this block is 14.24.74.207/28;
- If we add all addresses in the previous subblocks, the result is 208 addresses,
 - means 48 addresses are left in reserve.
- The first address in this range is 14.24.74.208.
- The last address is 14.24.74.255.
- We don't know about the prefix length yet.

IPv4 Addresses: Classless Addressing



IPv4 Addresses: Classless Addressing

An ISP is granted a block of addresses starting with 190.100.0.0/16 (65,536 addresses). The ISP needs to distribute these addresses to three groups of customers as follows:

- a. The first group has 64 customers; each needs 256 addresses.*
- b. The second group has 128 customers; each needs 128 addresses.*
- c. The third group has 128 customers; each needs 64 addresses.*

Design the subblocks and find out how many addresses are still available after these allocations.

IPv4 Addresses: Classless Addressing

Group 1

For this group, each customer needs 256 addresses. This means that 8 ($\log_2 256$) bits are needed to define each host. The prefix length is then $32 - 8 = 24$. The addresses are

<i>1st Customer:</i>	<i>190.100.0.0/24</i>	<i>190.100.0.255/24</i>
<i>2nd Customer:</i>	<i>190.100.1.0/24</i>	<i>190.100.1.255/24</i>
<i>...</i>		
<i>64th Customer:</i>	<i>190.100.63.0/24</i>	<i>190.100.63.255/24</i>
<i>Total = $64 \times 256 = 16,384$</i>		

Activate VM

IPv4 Addresses: Classless Addressing

Group 2

For this group, each customer needs 128 addresses. This means that 7 ($\log_2 128$) bits are needed to define each host. The prefix length is then $32 - 7 = 25$. The addresses are

<i>1st Customer:</i>	<i>190.100.64.0/25</i>	<i>190.100.64.127/25</i>
<i>2nd Customer:</i>	<i>190.100.64.128/25</i>	<i>190.100.64.255/25</i>
<i>...</i>		
<i>128th Customer:</i>	<i>190.100.127.128/25</i>	<i>190.100.127.255/25</i>
<i>Total = $128 \times 128 = 16,384$</i>		

IPv4 Addresses: Classless Addressing

Group 3

For this group, each customer needs 64 addresses. This means that 6 ($\log_2 64$) bits are needed to each host. The prefix length is then $32 - 6 = 26$. The addresses are

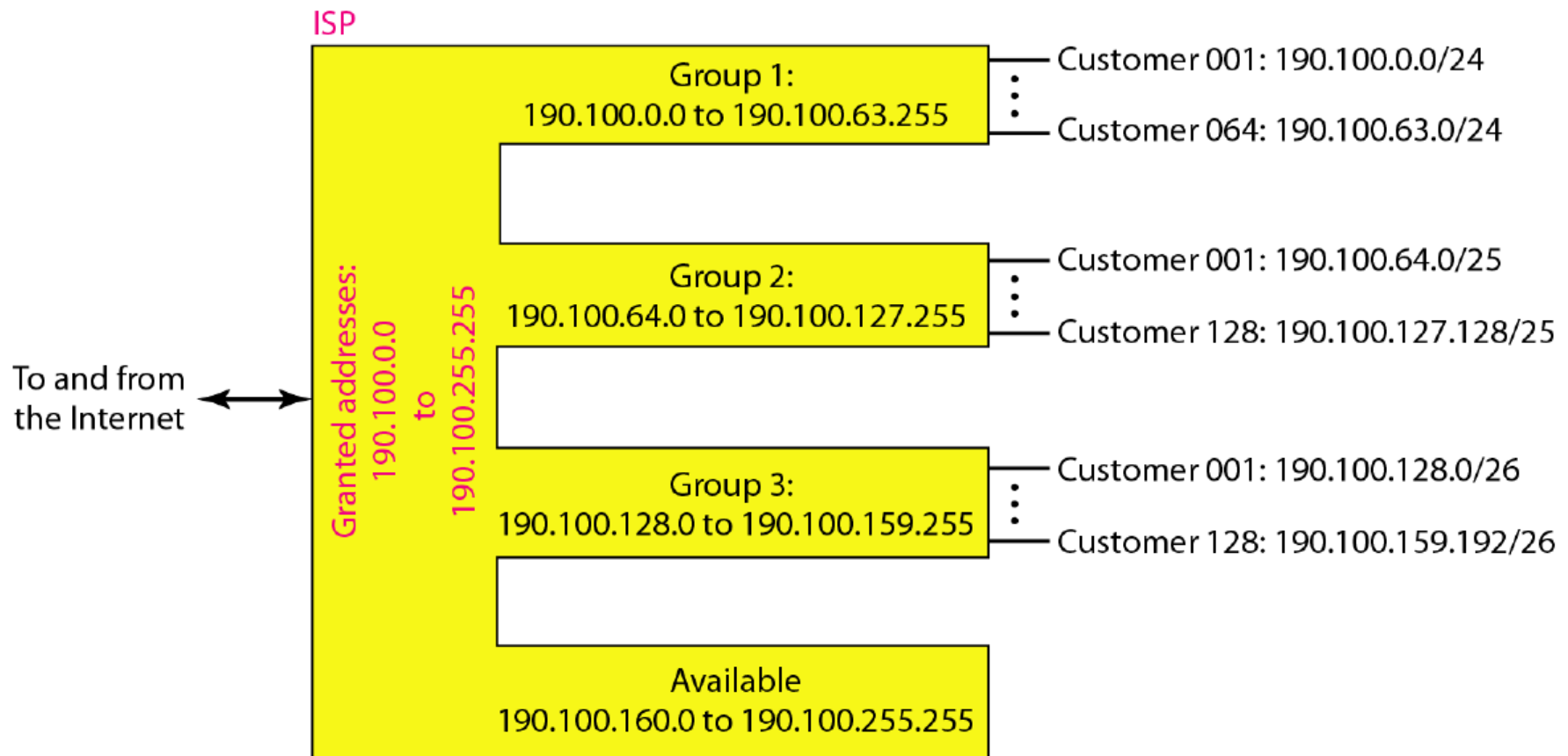
<i>1st Customer:</i>	<i>190.100.128.0/26</i>	<i>190.100.128.63/26</i>
<i>2nd Customer:</i>	<i>190.100.128.64/26</i>	<i>190.100.128.127/26</i>
<i>...</i>		
<i>128th Customer:</i>	<i>190.100.159.192/26</i>	<i>190.100.159.255/26</i>
<i>Total = $128 \times 64 = 8192$</i>		

Number of granted addresses to the ISP: 65,536

Number of allocated addresses by the ISP: 40,960

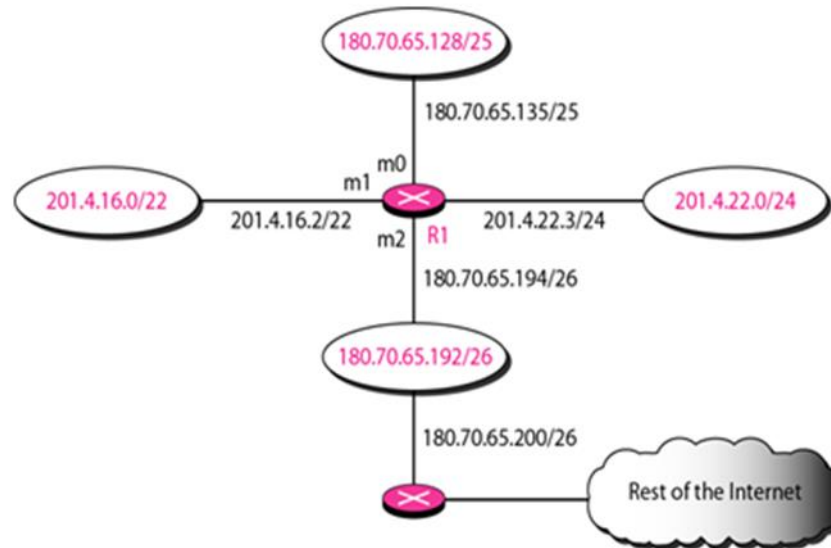
Number of available addresses: 24,576

IPv4 Addresses: Classless Addressing



IPv4 Addresses: Example Problems

1. Given the following network configuration, build a routing table for Router R1 consisting of mask, network address, next hop and interface. Illustrate the forwarding process if a packet arrives at R1 in with the destination address 201.4.22.35.



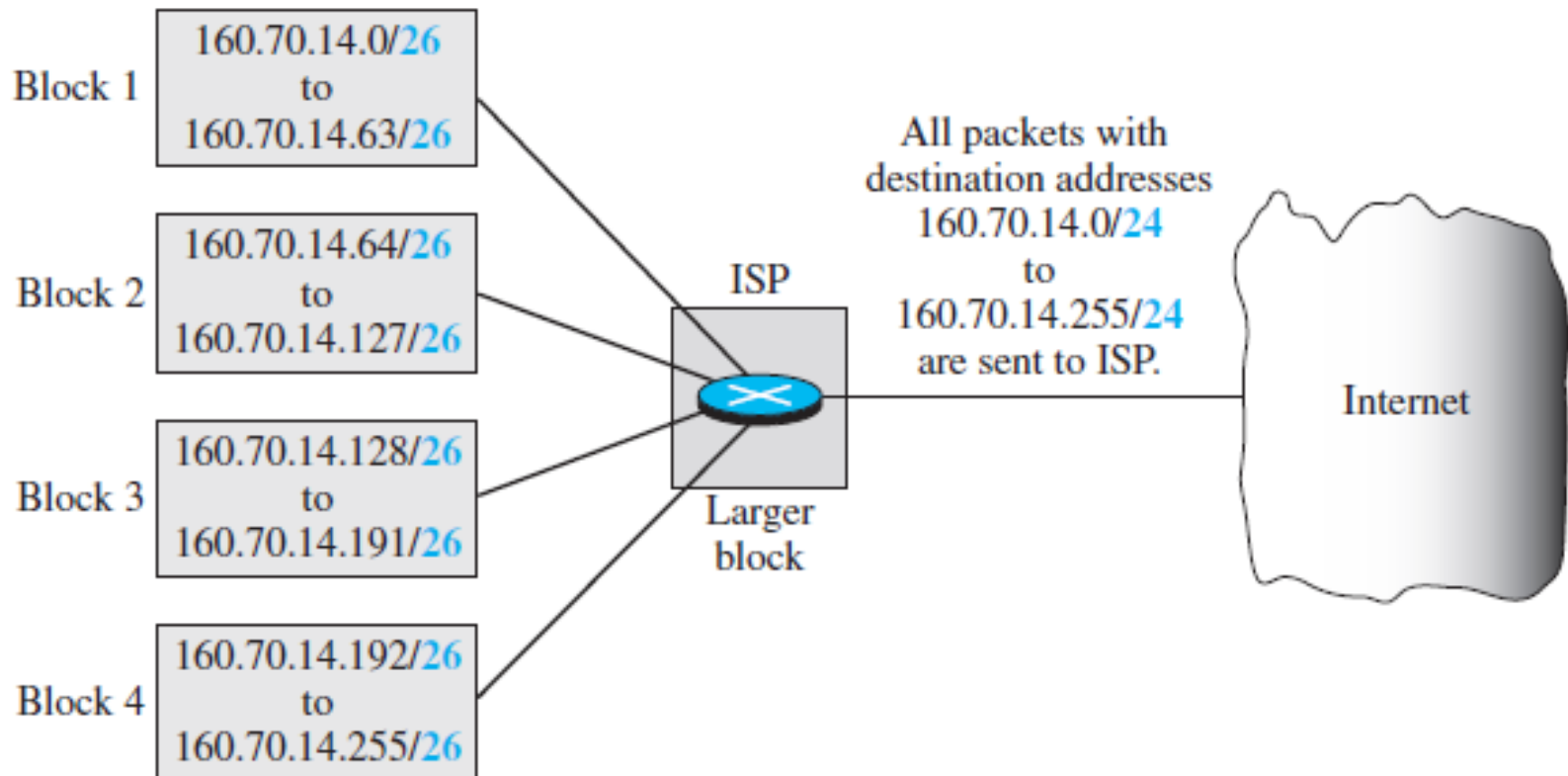
2. An organization is granted a block of addresses with the beginning address 194.2.2.0/24. The organization needs to have 3 subblocks of addresses to use in its three subnets: one subblock of 5 addresses, one subblock of 50 addresses, and one subblock of 130 addresses. Design the subblocks.

IPv4 Addresses: Address Aggregation

- **Address Aggregation**
- One of the advantages of the CIDR strategy is address aggregation (sometimes called address summarization or route summarization).
- When blocks of addresses are combined to create a larger block, **routing can be done based on the prefix of the larger block.**
- ICANN assigns a large block of addresses to an ISP.
- Each ISP in turn divides its assigned block into smaller subblocks and grants the subblocks to its customers.

IPv4 Addresses: Address Aggregation

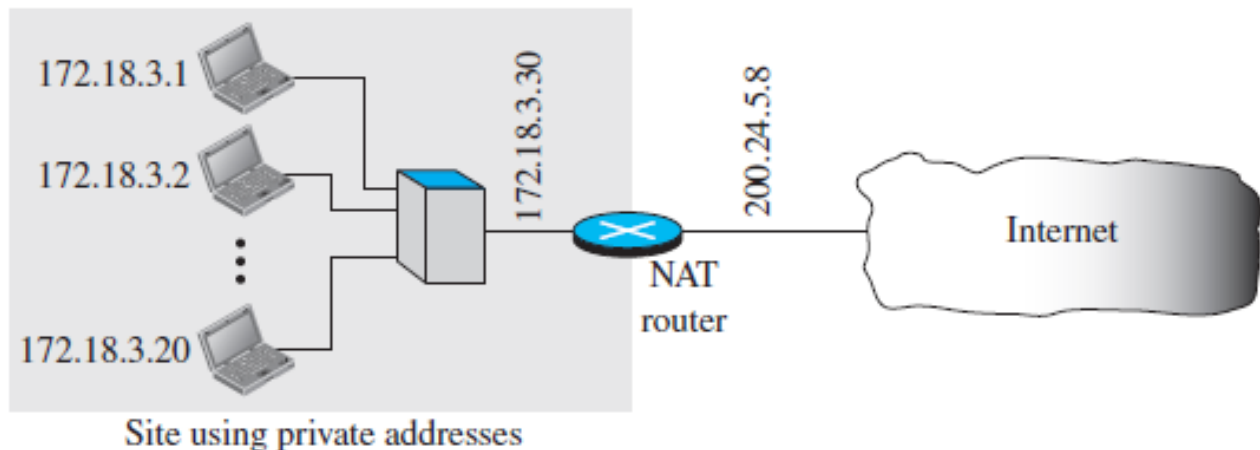
Figure 18.24 *Example of address aggregation*



Network Address Translation (NAT)

- The technology allows a site to use a **set of private addresses** for internal communication and a **set of global Internet addresses** (at least one) for communication with the rest of the world.
- The site must have only **one connection to the global Internet** through a NAT-capable router that runs NAT software. Figure 18.29 shows a simple implementation of NAT.
- The router that connects the network to the global address uses **one private address and one global address**.
- The private network is **invisible to the rest** of the Internet; the rest of the Internet sees only the NAT router with the address 200.24.5.8.

Figure 18.29 NAT

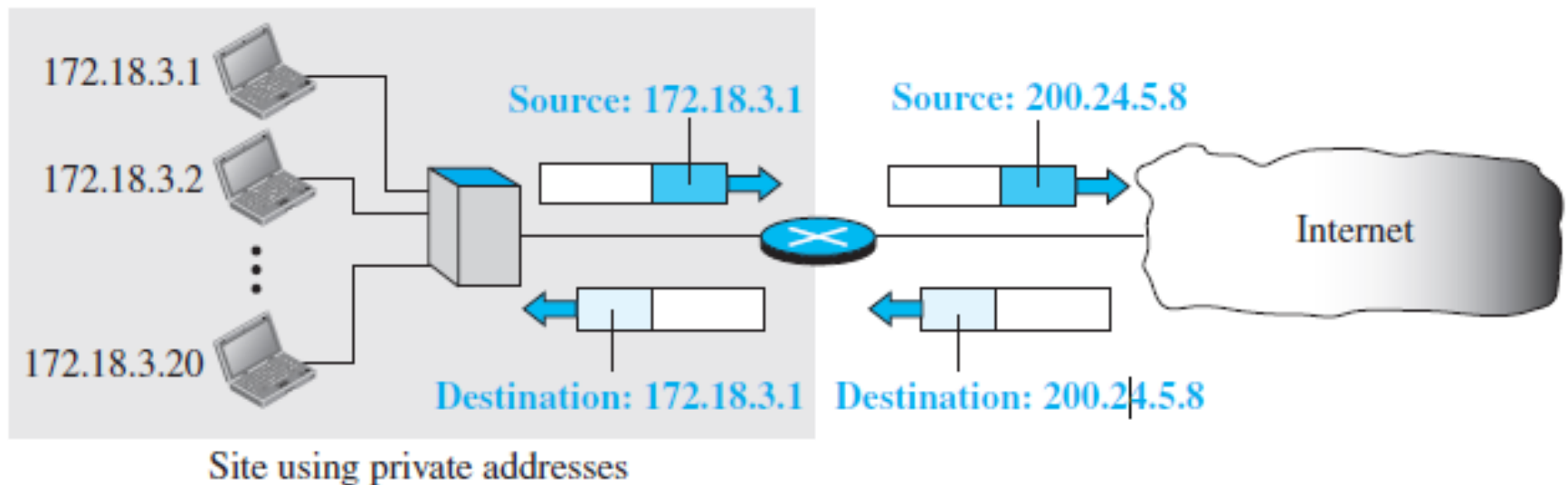


Network Address Resolution (NAT)

Address Translation

- All of the outgoing packets go through the NAT router, which replaces the source address in the packet with the global NAT address.
- All incoming packets also pass through the NAT router, which replaces the destination address in the packet (the NAT router global address) with the appropriate private address

Figure 18.30 *Address translation*



Network Address Resolution (NAT)

Translation Table

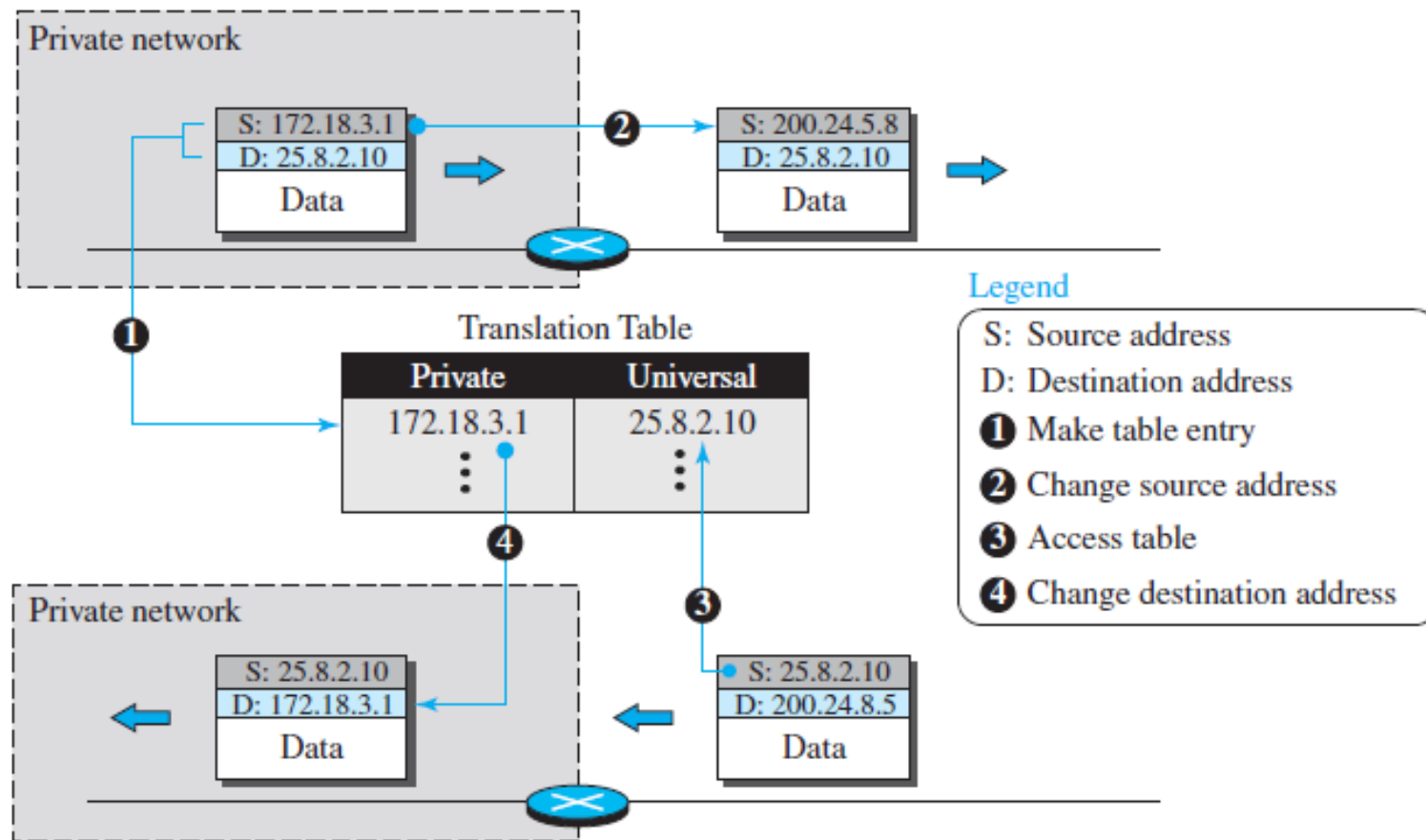
- Translating the source addresses for an outgoing packet is straightforward.
- But how does the NAT router know **the destination address for a packet coming from the Internet?** The problem is solved if the NAT router has a translation table.

1. Using One IP Address

- In its simplest form, a translation table has only two columns: the private address and the external address (destination address of the packet).
- When the router translates the source address of the outgoing packet, it also makes note of the destination address—where the packet is going.
- When the response comes back from the destination, the router uses the source address of the packet (as the external address) to find the private address of the packet.

Network Address Resolution (NAT)

Figure 18.31 Translation



Network Address Resolution (NAT)

2. Using a Pool of IP Addresses

- The NAT router can use four addresses (200.24.5.8, 200.24.5.9, 200.24.5.10, and 200.24.5.11).
- In this case, four private-network hosts can communicate with the same external host at the same time because each pair of addresses defines a separate connection

IPv6 Addresses

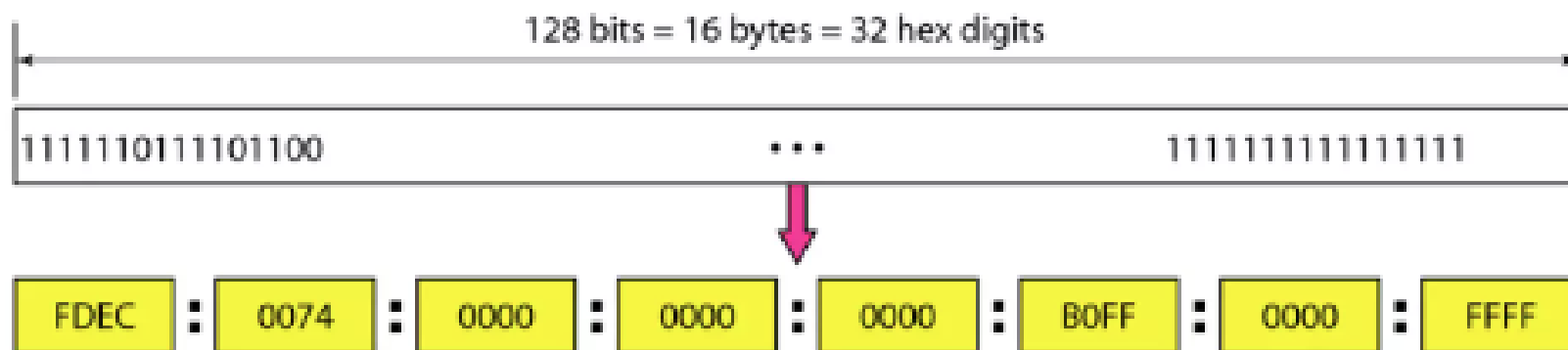
- Despite all short-term solutions, such as classless addressing, Dynamic Host Configuration Protocol (DHCP) and NAT, address depletion is still a long-term problem for the Internet.
- An IPv6 address consists of 16 bytes (octets); it is 128 bits long.

An IPv6 address is 128 bits long.

IPv6 Addresses

Hexadecimal Colon Notation

- To make addresses more readable, IPv6 specifies hexadecimal colon notation.
- **128 bits is divided into eight sections, each 2 bytes in length. Two bytes in hexadecimal notation requires four hexadecimal digits.**
- Therefore, the address consists of **32 hexadecimal digits**, with every four digits separated by a colon,

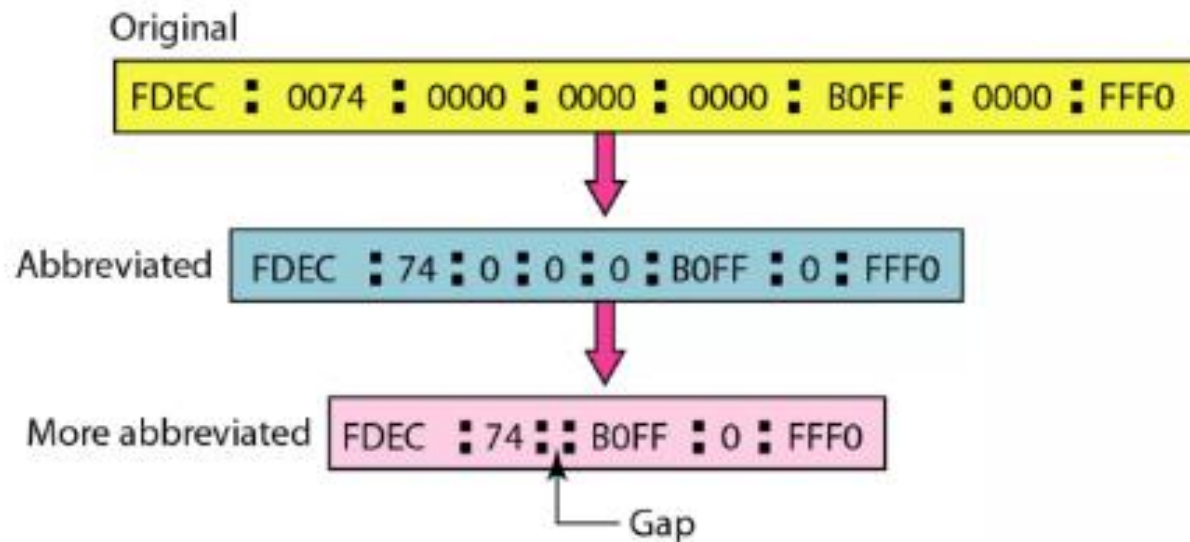


IPv6 Addresses

Although the IP address, even in hexadecimal format, is very **long, many of the digits are zeros.**

In this case, we can **abbreviate the address.** The leading zeros of a section (four digits between two colons) can be omitted.

Figure 19.15 *Abbreviated IPv6 addresses*



IPV6 Addresses

Expand the address 0:15::1:12:1213 to its original.

Solution

We first need to align the left side of the double colon to the left of the original pattern and the right side of the double colon to the right of the original pattern to find how many 0s we need to replace the double colon.

XXXX:XXXX:XXXX:XXXX:XXXX:XXXX:XXXX:XXXX
0: 15: : 1: 12:1213

This means that the original address is.

0000:0015:0000:0000:0000:0001:0012:1213

Internet Protocol (IP)

- **Unreliable datagram protocol**—a best-effort delivery service.
- **Best effort** means that IPv4 packets can be corrupted, be lost, arrive out of order, be delayed or create congestion for the network.
- **Example: Post Office**
- If **reliability** is important, IPv4 must be paired with a reliable transport-layer protocol such as TCP.
- IPv4 is also a **connectionless** protocol: Each datagram is handled independently, and each datagram can follow a different route to the destination. Datagrams sent by the same source to the same destination could arrive out of order.

Internet Protocol (IP)

- First service provided by IPv4 – packetizing
- IPv4 format of a packet in which the data coming from the upper layer or other protocols are encapsulated.
- Packets used by the IP are called **datagrams**.
- A datagram is a variable-length packet consisting of **two parts**:

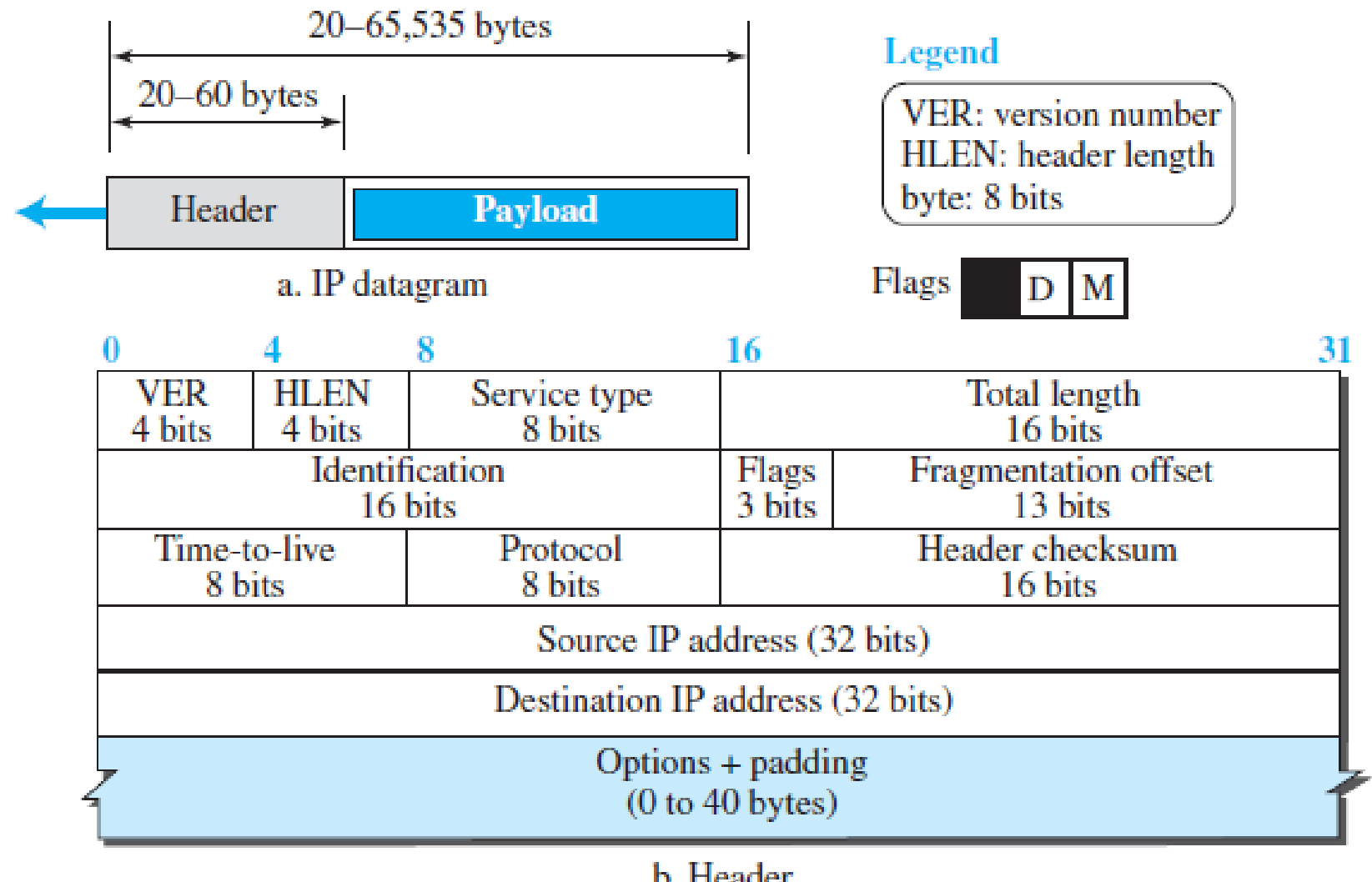
1. Header

- The header is 20 to 60 bytes in length and contains information essential to routing and delivery
- First 20 bytes are essential and together are called the main header next 40 bytes include options and padding that may or may not be present
- It is customary in TCP/IP to show the header in 4-byte sections.

2. Payload (data).

Internet Protocol (IP)

Figure 19.2 IP datagram



IPv4 Datagram Format

- **Version** specifies the IP version.
- **Header length (HLEN)** specifies the length of the header (including options and padding) in terms of 4-byte blocks. For example, if the total header of a packet (including options and padding) is 60B, $HL=60B/4B=15$ which is 1111 in binary.
- **Type of service** specifies the quality-of-service (QoS) requirements of the packet, such as priority level, delay, reliability, throughput, and cost.
- **Total length** specifies the total length of the packet in bytes, including the header and data. A total of 16 bits are assigned to this field. $\text{Length of data} = \text{total length} - (\text{HLEN}) \times 4$
 - Helps to determine how much is really data and how much is padding
- **Identification, flags, and fragment offset** are used for packet fragmentation and reassembly.

IPv4 Datagram Format

- ***Time to live* specifies the maximum number of hops after which a packet must be discarded.**
 - value is approximately 2 times the maximum number of routes between any two hosts.
 - Each router that processes the datagram decrements this number by one.
 - If this value, after being decremented, is zero, the router discards the datagram.
- ***Protocol* specifies the protocol of packet in payload.**
 - A datagram can carry a packet from other protocols that directly use the service of the IP, such as some routing protocols, any transport-layer protocol or some auxiliary protocols.
 - provides multiplexing at the source and demultiplexing at the destination,

IPv4 Datagram Format

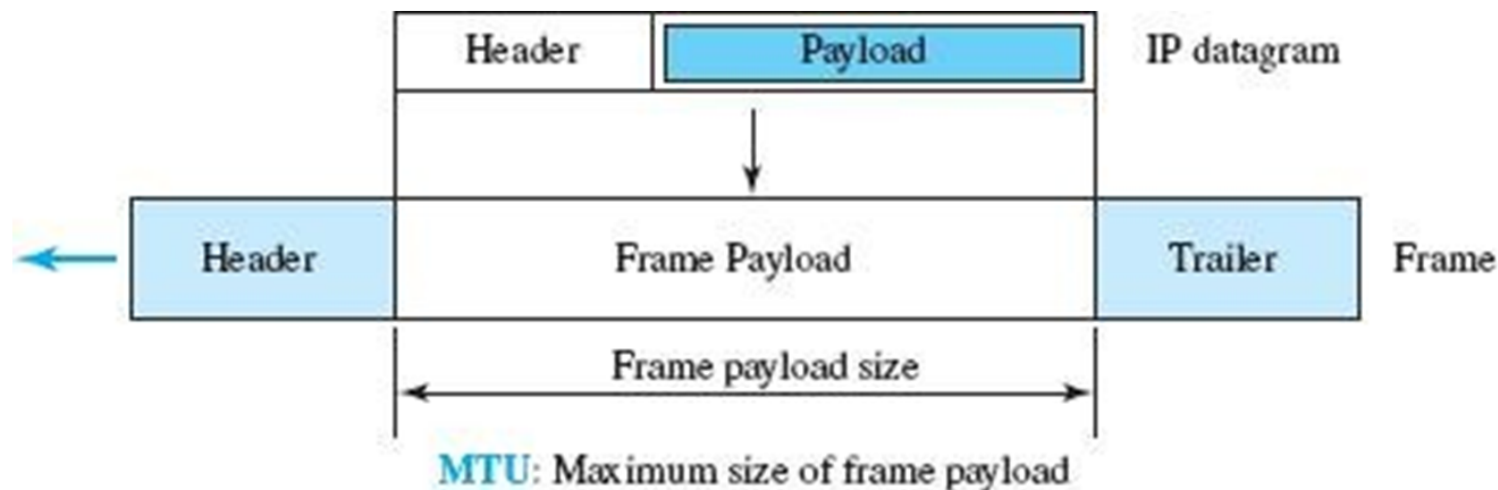
- **Header checksum** is a method of error detection
 - the value of some fields, such as TTL, which are related to fragmentation and options, may change from router to router, the checksum needs to be recalculated at each router
- **Source address and destination address** are 32-bit fields specifying the source address and the destination address, respectively.
- remain unchanged during the time the IP datagram travels from the source host to the destination host.
- **Options** is a rarely used variable-length field to specify security level, timestamp, and type of route.
- used for network testing and debugging
- **Padding** is used to ensure that the header is a multiple of 32 bits.

Fragmentation

- Datagram can travel through **different networks**.
- Each router decapsulates the IP datagram from the frame it receives, processes it, and then encapsulates it in another frame.
- The **format and size of the received frame** depend on the protocol used by the physical network through which the frame has just traveled.
- For example, if a router connects **a LAN to a WAN**, it receives a frame in the LAN format and sends a frame in the WAN format.

Fragmentation

- Each link-layer protocol has its **own frame format**.
- When a datagram is encapsulated in a frame, the total size of the datagram must be less than this maximum size, which is defined by the restrictions imposed by the hardware and software used in the network.



Fragmentation

- The value of the maximum transfer unit (MTU) differs from one physical network protocol to another.
- For example, the value for a **LAN is normally 1500** bytes, but for a WAN it can be larger or smaller.
- Need a mechanism that avoids requiring **large buffers** at intermediate routers to store the fragments.
- To make the IP protocol independent of the physical network, the designers decided to make the **maximum length of the IP datagram equal to 65,535 bytes**.
- However, for other physical networks, we must **divide the datagram** to make it possible for it to pass through these networks. This is called **fragmentation**.
 - **Payload** of the IP datagram is fragmented
 - Most parts of the **header**, with the exception of some options, must be copied by all fragments.

Fragmentation

- A fragmented datagram may itself be fragmented if it encounters a network with an even smaller MTU.
- A datagram can be fragmented by the source host or any router in the path.
- The **reassembly of the datagram**, is done only by the destination host, because each fragment becomes an independent datagram.
- Fragmented datagram can travel through different routes, all the fragments belonging to the same datagram should finally arrive at the destination host.

Fragmentation

- The host or router that fragments a datagram must change the values of three fields: **flags, fragmentation offset, and total length**.
- The rest of the fields must be copied.
- The value of the checksum must be recalculated regardless of fragmentation.
- The identification field identifies a datagram originating from the source host.
 - **Counter** is initialized to a positive number
 - When a datagram is fragmented, the value in the **identification field is copied into all fragments**
 - Identification number helps the destination in **reassembling**
 - The **offset** field indicates the **position** of a fragment in the sequence of fragments making up the packet.

Fragmentation

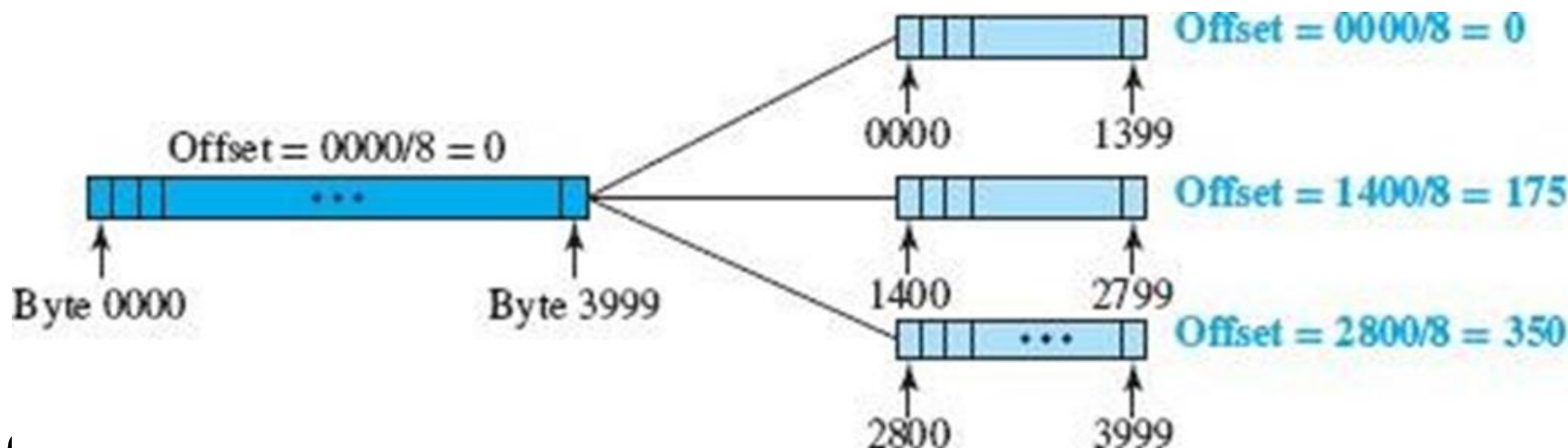
- The 3-bit flags field defines three flags.
 1. The leftmost bit is **reserved (not used)**.
 2. The second bit (D bit) is called the **do not fragment bit**.
 - 1: The machine must not fragment the datagram.
 - 0: The datagram can be fragmented if necessary.
 3. The third bit (M bit) is called the **more fragment bit**.
 - 1: The datagram is not the last fragment; there are more fragments after this one.
 - 0: This is the last or only fragment.

Fragmentation

- The 13-bit fragmentation offset field shows the relative position of this fragment with respect to the whole datagram.
- It is the offset of the data in the original datagram measured in units of 8 bytes.
- Datagram with a data size of 4000 bytes (numbered 0 to 3999) fragmented into three fragments.
- The first fragment carries bytes 0 to 1399. The offset for this datagram is $0/8 = 0$.
- The second fragment carries bytes 1400 to 2799; the offset value for this fragment is $1400/8 = 175$.
- Finally, the third fragment carries bytes 2800 to 3999. The offset value for this fragment is $2800/8 = 350$.

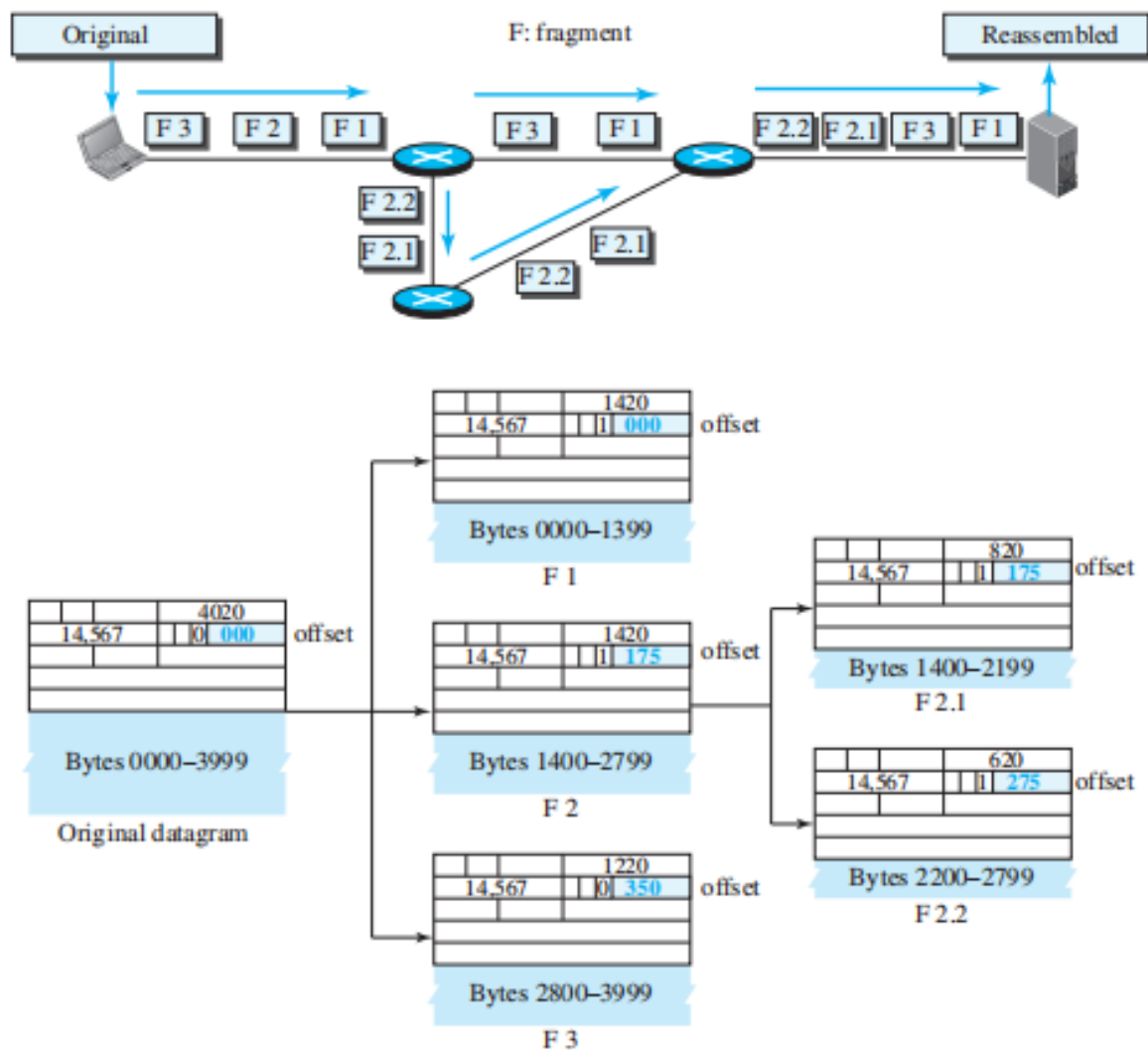
Fragmentation

- The 13-bit fragmentation offset field shows the relative position of this fragment with respect to the whole datagram.
- It is the offset of the data in the original datagram measured in units of 8 bytes.
- **Example:** Datagram with a data size of 4000 bytes (numbered 0 to 3999) fragmented into three fragments.
 - The first fragment carries bytes 0 to 1399. The offset for this datagram is $0/8 = 0$.
 - The second fragment carries bytes 1400 to 2799; the offset value for this fragment is $1400/8 = 175$.
 - Finally, the third fragment carries bytes 2800 to 3999. The offset value for this fragment is $2800/8 = 350$.



Fragmentation

- The original packet starts at the client; the fragments are reassembled at the Server
- M bit set for all fragments except the last
- the second fragment is itself fragmented later into two fragments of 800 bytes and 600 bytes, but the offset shows the relative position of the fragments to the original data.



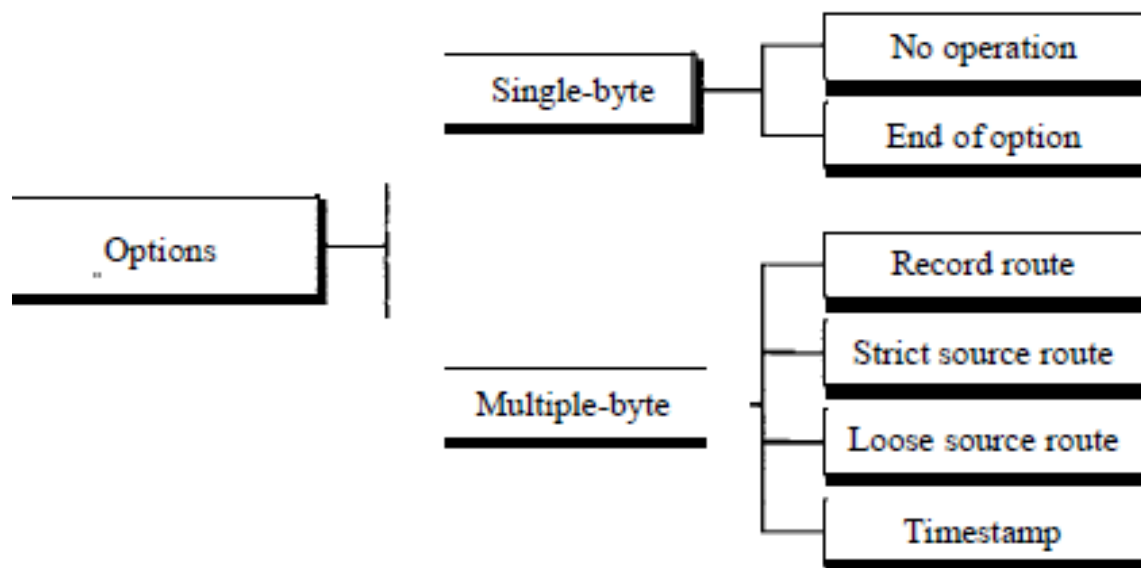
Options

- The header of the IPv4 datagram is made of **two parts**:
 - Fixed part
 - Variable part.
- The **fixed part is 20 bytes** long and was discussed in the previous section.
- The **variable part** comprises the options that can be a maximum of 40 bytes.
- Options are divided into two broad categories:
 - **Single-byte options**
 - **Multiple-byte options.**

Options

- **Single-Byte Options** : There are two single-byte options.
 1. No Operation
 - A no-operation option is a 1-byte option used as a filler between options.
 2. End of Option
 - An end-of-option option is a 1-byte option used for padding at the end of the option field.

Taxonomy of options in IPv4



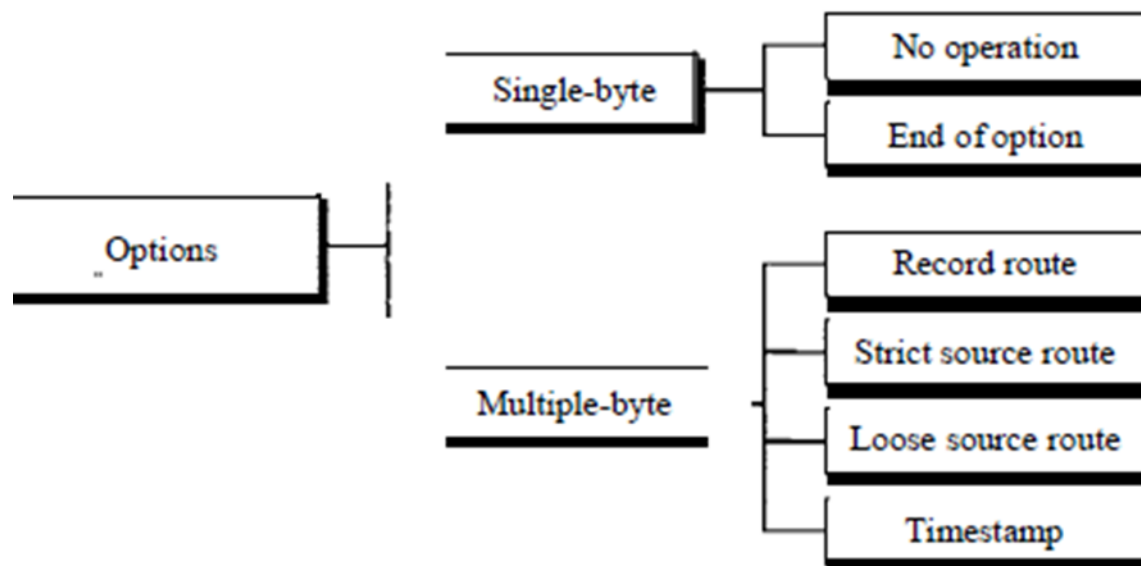
Options

- **Multiple-Byte Options** : There are four multiple-byte options.

1. Record Route

- A record route option is used to record the Internet routers that handle the datagram. It can list up to nine router addresses. It can be used for debugging and management purposes.

Taxonomy of options in IPv4



Options

2. Multiple-Byte Options : There are four multiple-byte options.

2. Strict Source Route

- A strict source route option is used by the source to predetermine a route for the datagram as it travels through the Internet.
- The sender can choose a route with a specific type of service, such as minimum delay or maximum throughput, a route that is safer or more reliable for the sender's purpose.
- All the routers defined in the option must be visited by the datagram.

3. Loose Source Route

- A loose source route option is similar to the strict source route, but it is less rigid.
- Each router in the list must be visited, but the datagram can visit other routers as well.

4. Timestamp

- A timestamp option is used to record the time of datagram processing by a router. The time is expressed in milliseconds

IPv6 Protocol

IPv4 has some deficiencies that make it unsuitable for the fast-growing Internet.

1. Despite all short-term solutions, such as subnetting, classless addressing, and NAT, **address depletion** is still a long-term problem in the Internet.
2. The Internet must accommodate real-time audio and video transmission. This type of transmission requires minimum delay strategies and **reservation of resources** not provided in the IPv4 design.
3. The Internet must accommodate encryption and authentication of data for some applications. No **encryption or authentication** is provided by IPv4.

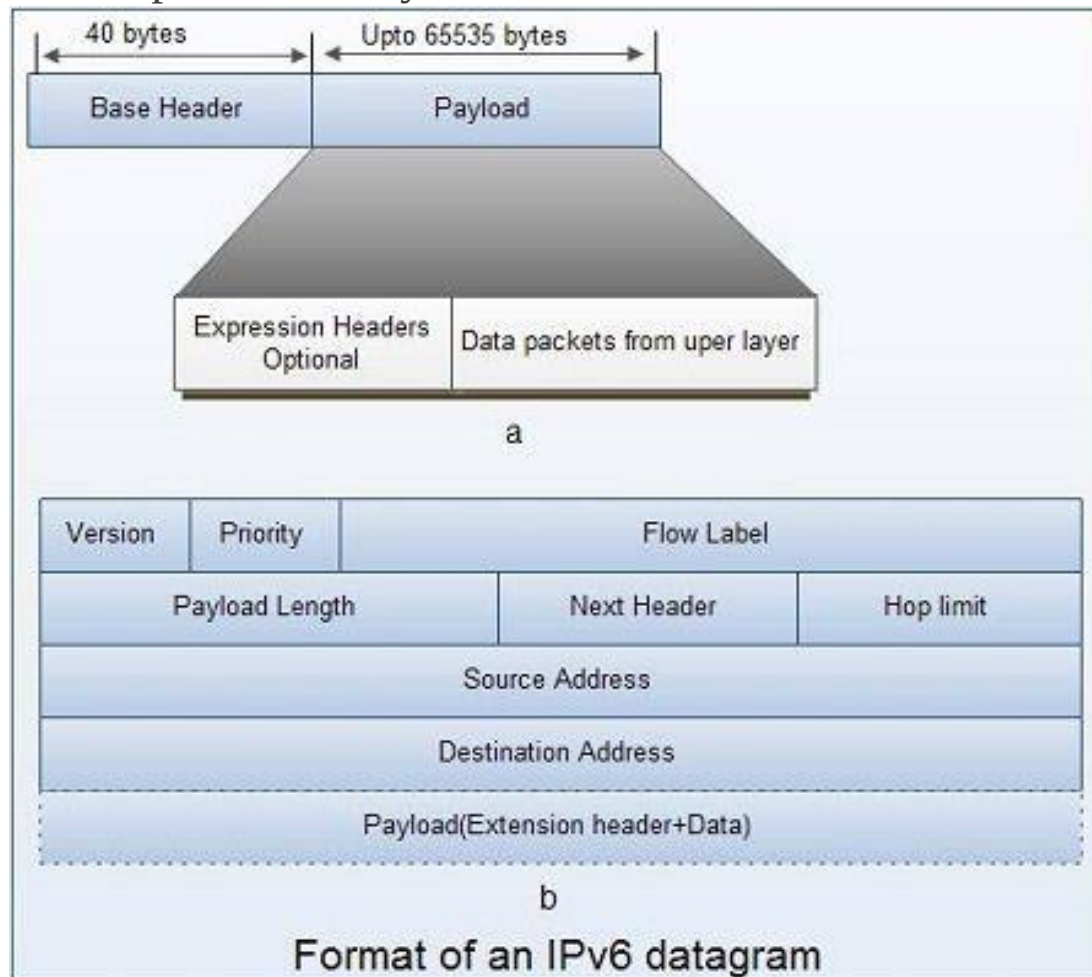
To overcome these deficiencies, **IPv6 (Internetworking Protocol, version 6)**, also known as **IPng (Internetworking Protocol, next generation)**, was proposed and is now a standard.

IPv6 Advantages

- **Larger address space.** An IPv6 address is 128 bits long.
- **Better header format.** IPv6 uses a new header format in which options are separated from the base header and inserted, when needed, between the base header and the upper-layer data. This simplifies and speeds up the routing process because most of the options do not need to be checked by routers.
- **New options.** IPv6 has new options to allow for additional functionalities.
- **Allowance for extension.** IPv6 is designed to allow the extension of the protocol if required by new technologies or applications.
- **Support for resource allocation.** In IPv6, the type-of-service field has been can be used to support traffic such as real-time audio and video.
- **Support for more security.** The encryption and authentication options in IPv6 provide confidentiality and integrity of the packet.

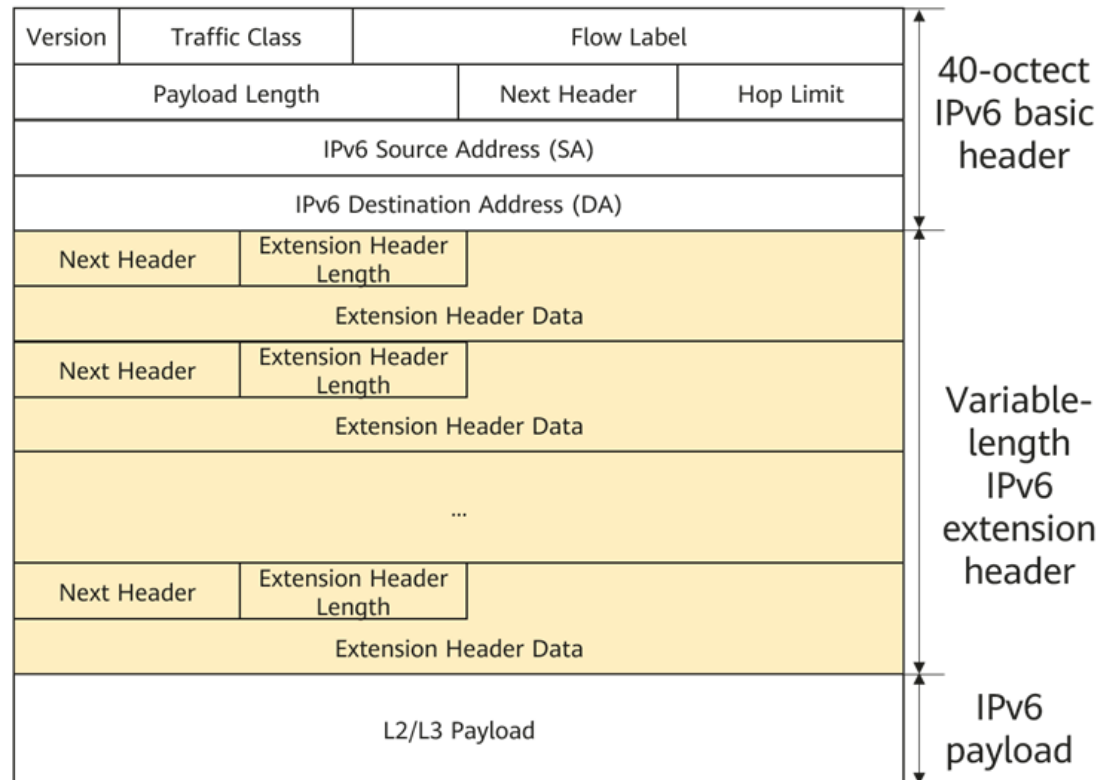
IPv6 Packet Format

- Each packet is composed of a **mandatory base header followed by the payload**. The payload consists of two parts: optional extension headers and data from an upper layer.
- The base header occupies 40 bytes, whereas the extension headers and data from the upper layer contain up to 65,535 bytes of information.



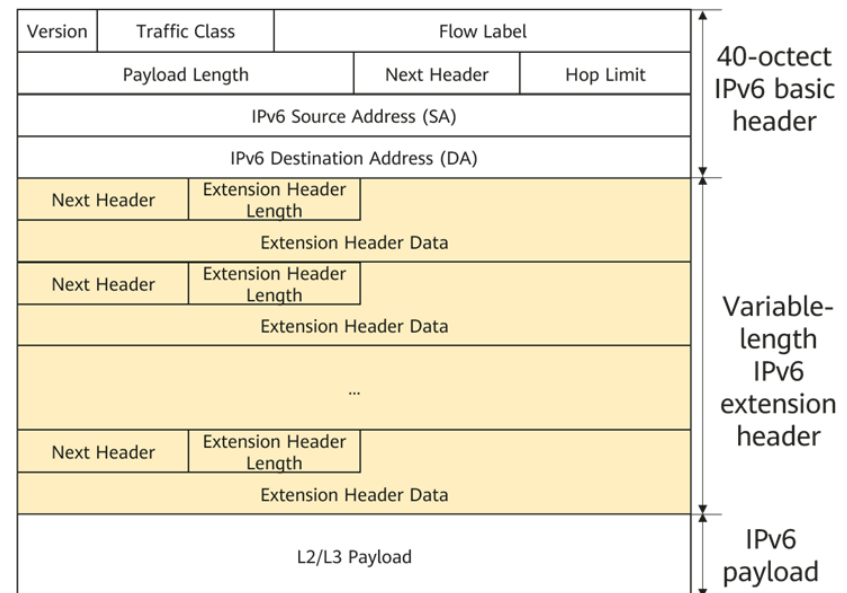
IPv6 Packet Format

- **Base Header :** The base header has eight fields.
- These fields are as follows:
- **Version.** This 4-bit field defines the version number of the IP. For IPv6, the value is 6.
- **Priority.** The 4-bit priority field defines the priority of the packet with respect to traffic congestion.
- Flow label. The flow label is a 3-byte (24-bit) field that is designed to provide special handling for a particular flow of data.



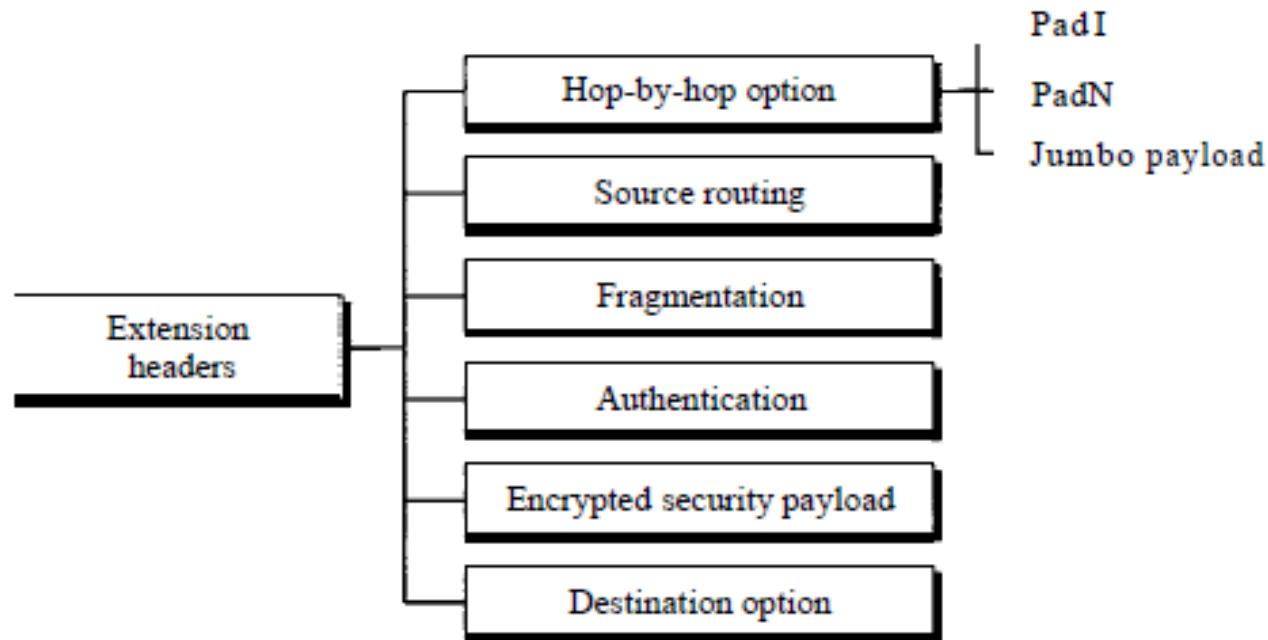
IPv6 Packet Format

- **Payload length.** The 2-byte payload length field defines the length of the IP datagram excluding the base header.
- **Next header.** The next header is an 8-bit field defining the header that follows the base header in the datagram. The next header is either one of the optional extension headers used by IP or the header of an encapsulated packet such as UDP or TCP. Each extension header also contains this field.
- **Hop limit.** This 8-bit hop limit field serves the same purpose as the TTL field in IPv4.
- **Source address.** The source address field is a 16-byte (128-bit)
- **Destination address.** The destination address field is a 16-byte (128-bit)



IPv6 Extension Headers

- The length of the base header is fixed at 40 bytes.
- To give greater functionality to the IP datagram, the base header can be followed by up to six extension headers.
- Six types of extension headers have been defined



IPv6 Extension Headers

- **Hop-by-Hop Option**
 - The hop-by-hop option is used when the source needs to pass information to all routers visited by the datagram.
 - Three options have been defined: Padl, PadN, and jumbo payload.
 - The Padl option is 1 byte long and is designed for alignment purposes.
 - PadN is similar in concept to Padi. The difference is that PadN is used when 2 or more bytes is needed for alignment.
 - The jumbo payload option is used to define a payload longer than 65,535 bytes.
- **Source Routing**
 - The source routing extension header combines the concepts of the strict source route and the loose source route options of IPv4.
- **Fragmentation**
 - In IPv6, only the original source can fragment. A source must use a path MTU discovery technique to find the smallest MTU supported by any network on the path. The source then fragments using this knowledge.

IPv6 Extension Headers

- **Authentication**
 - The authentication extension header has a dual purpose: it validates the message sender and ensures the integrity of data.
- **Encrypted Security Payload**
 - The encrypted security payload (ESP) is an extension that provides confidentiality and guards against eavesdropping.
- **Destination Option**
 - The destination option is used when the source needs to pass information to the destination only. Intermediate routers are not permitted access to this information.

Comparison Between IPv4 and IPv6 Headers

Table 20.9 compares IPv4 and IPv6 headers.

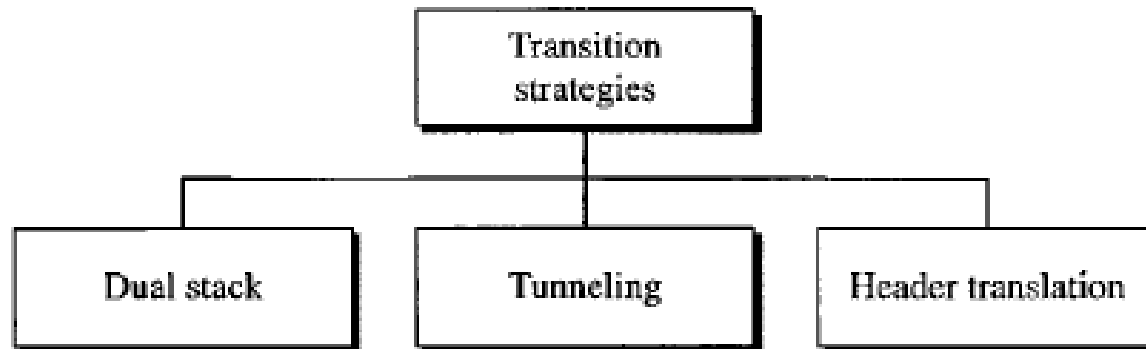
Table 20.9 *Comparison between IPv4 and IPv6 packet headers*

<i>Comparison</i>
1. The header length field is eliminated in IPv6 because the length of the header is fixed in this version.
2. The service type field is eliminated in IPv6. The priority and flow label fields together take over the function of the service type field.
3. The total length field is eliminated in IPv6 and replaced by the payload length field.
4. The identification, flag, and offset fields are eliminated from the base header in IPv6. They are included in the fragmentation extension header.
5. The TTL field is called hop limit in IPv6.
6. The protocol field is replaced by the next header field.
7. The header checksum is eliminated because the checksum is provided by upper-layer protocols; it is therefore not needed at this level.
8. The option fields in IPv4 are implemented as extension headers in IPv6.

Transition From IPV4 To IPV6

- Because of the huge number of systems on the Internet, the transition from IPv4 to IPv6 **cannot happen suddenly**.
- It takes a considerable **amount of time** before every system in the Internet can move from IPv4 to IPv6.
- The transition must **be smooth to prevent** any problems between IPv4 and IPv6 systems.
- **Three strategies** have been devised by the IETF to help the transition

Three transition strategies



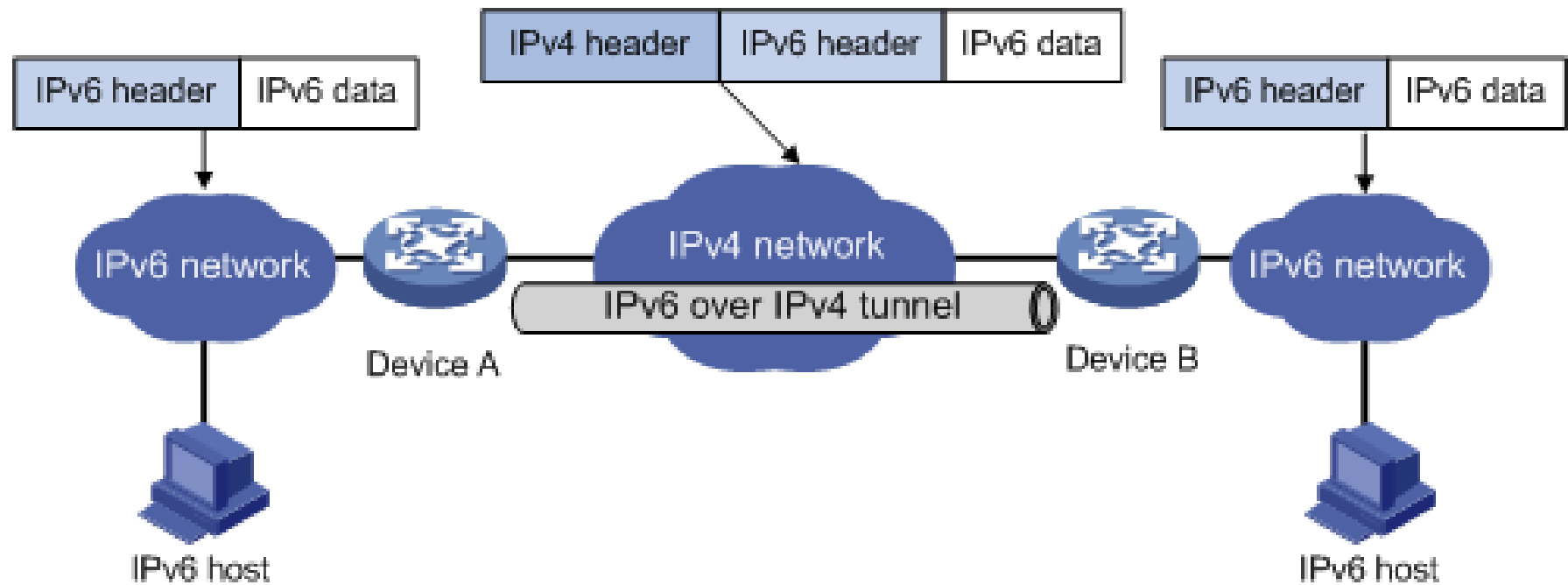
Transition From IPV4 To IPV6: Dual Stack Protocol

- It is recommended that all hosts, before migrating completely to version 6, have a dual stack of protocols.
- A station must run IPv4 and IPv6 simultaneously until all the Internet uses IPv6.
- To determine which version to use when sending a packet to a destination, the source host queries the DNS.
- If the DNS returns an IPv4 address, the source host sends an IPv4 packet.
- If the DNS returns an IPv6 address, the source host sends an IPv6 packet.

Transition From IPV4 To IPV6: Tunneling

- Tunneling is a strategy used when two computers using IPv6 want to communicate with each other and the packet must pass through a region that uses IPv4.
- To pass through this region, the packet must have an IPv4 address.
- IPv6 packet is encapsulated in an IPv4 packet when it enters the region, and it leaves its capsule when it exits the region.
- It seems as if the IPv6 packet goes through a tunnel at one end and emerges at the other end.
- To make it clear that the IPv4 packet is carrying an IPv6 packet as data, the protocol value is set to 41.

Transition From IPV4 To IPV6: Tunneling



Transition From IPV4 To IPV6: Header Translation

- Header translation is necessary when the majority of the Internet has moved to IPv6 but some systems still use IPv4. The sender wants to use IPv6, but the receiver does not understand IPv6.
- The header format must be totally changed through header translation.

